

CHUVADI

सुवडी



Chuvadi.Sheets

The Complete Guide

A pure-BCL spreadsheet and zip library for .NET



VERSION

1.0

AUTHOR

Arun Shivab

YEAR

2026

Chuvadi.Sheets — The Complete Guide

Version 1.0 documentation, May 2026.

Library © 2026 Arun Shivab. Released under the MIT License.

Documentation © 2026 Arun Shivab.

This document and the library it describes are part of the *Chuvadi* family of .NET libraries, named after the palm-leaf manuscript (சுவடி) — a tradition of compact, self-contained, durable knowledge.

"Chuvadi.Sheets" and the LiPi (लिपि) typography aesthetic are trade dress identifying this library family.

Source: github.com/arunshivab/chuvadi

Contents

FRONT MATTER

Preface	xi
How to read this guide	xii

PART I · GETTING STARTED

1 Introduction	3
1.1 What Chuvadi.Sheets is	3
1.2 Why another xlsx library	5
1.3 Feature overview	7
1.4 What it does not do	9
2 Installation	11
2.1 From a local NuGet folder	11
2.2 From GitHub Packages	13
2.3 From nuget.org	14
3 First example	15
4 Library layout	19

PART II · ARCHITECTURE

5 Design principles	23
6 Module map	26
7 Write pipeline	28
8 Read pipeline	30
9 Encrypted file format	32

PART III · WRITING XLSX FILES

10 The two write APIs	37
11 Workbook model	39
12 Streaming writer	46
13 Cells, values, types	52
14 Cell styling	58
15 Ergonomic exports	63
16 Sheet features	68
17 Data validation	72
18 Conditional formatting	75
19 Defined names	78

PART IV · READING XLSX FILES

20 Streaming reader	83
21 Loading a workbook	89
22 Ergonomic reads	92

23	Cell value extraction	95
<i>PART V · PROTECTION AND ENCRYPTION</i>		
24	Sheet protection	101
25	Workbook structure protec-	
26	Workbook encryption	106
27	Security model	110
<i>PART VI · ZIP UTILITIES</i>		
28	ZipWriter	115
29	ZipReader	119
30	Convenience extensions	122
<i>PART VII · PERFORMANCE AND OPERATIONS</i>		
31	Performance characteristics	127
32	Streaming vs random-ac-	
33	Thread safety and concur-	
<i>PART VIII · REFERENCE</i>		
34	Public API summary	137
35	Exception types	147
36	Options and configuration	149
<i>APPENDICES</i>		
A	Complete example — patient re-	
B	Complete example — financial re-	
C	Complete example — encrypted	
D	Complete example — CSV to xlsx	
E	Troubleshooting	172
F	Changelog	175
G	License	176

Preface

A short note about why this library exists, what it tries to do well, and what it is content to leave out.

Spreadsheet files are the lingua franca of business data. They move between departments, between systems, between people who would never agree on a database schema. For any application that writes reports, accepts customer-submitted data, exports tables, or shares results outside its own walls, the ability to produce and consume `.xlsx` files is not an optional feature — it is part of the cost of doing business.

For .NET developers, the established choices in this space are mature and capable. They are also, in different ways, heavy. Some carry dozens of transitive dependencies. Some require commercial licensing past trivial use. Some expose the entire OOXML schema as their public API and ask the caller to know it. Many produce files that work in Excel but not in alternative consumers, or vice versa.

Chuvadi.Sheets exists to occupy a different point in that space:

- **Pure .NET BCL.** No NuGet dependencies. No native binaries. Nothing to vet, nothing to upgrade independently. The library's own `.nupkg` declares no dependencies — you can verify this in the package metadata.
- **Small.** Roughly five hundred kilobytes of source, organised into a flat namespace. A new contributor can read the whole codebase in an afternoon.
- **Excel-correct.** The encryption path, in particular, has been validated against Microsoft Excel itself and against the spec-strict `msoffcrypto-tool` reference implementation. Files produced by this library open in Excel exactly the way files produced by Excel do.
- **Predictable.** Streaming writes use bounded memory regardless of dataset size. Streaming reads use a single reusable buffer per row, with zero allocations in the steady state. Operations behave the same way the tenth time you do them as the first.

It is also content to be incomplete. Charts, pivot tables, embedded images, rich text cells, and macros are deferred indefinitely. If those features are critical to your use case, this library is not for you, and that is fine. The point of Chuvadi.Sheets is to do a few things — read, write, style, validate, protect, encrypt — and do them with the precision and predictability of the `System.IO` primitives it sits on.

This guide is the reference for that contract. It is exhaustive on what the library does, honest about what it does not, and structured so a developer who has never touched the OOXML format can become productive within a chapter or two. The shape of the document follows the shape of the API: the early chapters introduce the model, the middle chapters work through every public type, and the later chapters and appendices show realistic uses in full programs.

If you read just one section before writing your first line of code, read Chapter 3.

How to read this guide

The guide is divided into eight parts and a set of appendices. Each part stands largely on its own, with cross-references where they help.

Part I — Getting started

The fastest path to a working program. Skim if you are already comfortable with .NET package management; read Chapter 3 in any case for the canonical hello-world.

Part II — Architecture

The shape of the library, with diagrams. Worth reading once, particularly Chapter 9 if you intend to use the encryption features.

Part III — Writing `xlsx` files

The largest part of the guide. Treat it as a reference; you do not need to read it linearly. Each chapter is self-contained.

Part IV — Reading `xlsx` files

Mirrors Part III but for the read direction. Substantially shorter because reading is mostly the inverse of writing.

Part V — Protection and encryption

Important for any application that handles regulated or confidential data. Chapter 27 sets out the security model explicitly.

Part VI — Zip utilities

A clean public API over `System.IO.Compression`. Useful in its own right; can be read without Parts III–IV.

Part VII — Performance and operations

Benchmarks, memory characteristics, and guidance on choosing between the streaming and random-access APIs.

Part VIII — Reference

A condensed catalogue of every public type and member. Useful as a lookup; the deeper explanations live in the body chapters.

Appendices

Four complete runnable example programs, troubleshooting notes, the changelog, and the license.

Conventions

Code blocks are tagged with the language they are written in (C#, shell, XML). The tag appears in the upper-left corner of the block.

NOTE

A blue-bordered box like this one is an aside — supplementary detail, not strictly required to follow the main text.

TIP

A gold-bordered box is a practical recommendation that can save time.

WARNING

A vermillion-bordered box is a caution — something that will go wrong if ignored.

SPECIFICATION

A khaki box references a relevant section of the OOXML or MS-OFFCRYPTO specification, for readers who want to verify the implementation against the source.

Code conventions match the Microsoft C# guidelines, with one local convention: when a code sample omits namespace imports for brevity, assume `using Chuvadi.Sheets.Excel;` and where relevant `using Chuvadi.Sheets.Zip;`.

References to types follow .NET conventions: `Workbook` for unqualified type names, `Chuvadi.Sheets.Excel.Workbook` for the full form, and `Workbook.SaveTo` for member references.



Part I

Getting started



1 • Introduction

1.1 What Chuvadi.Sheets is

Chuvadi.Sheets is a .NET library that reads and writes Excel `.xlsx` files, with optional workbook-level encryption and a clean zip API alongside. It is distributed as a single NuGet package, `Chuvadi.Sheets`, with no transitive dependencies of its own.

The library exposes three primary namespaces:

Chuvadi.Sheets.Excel

The `xlsx` surface — reading, writing, the workbook model, cell styling, sheet features, data validation, conditional formatting, sheet protection, and workbook encryption.

Chuvadi.Sheets.Zip

The `zip` surface — creating, extracting, and inspecting zip archives. Built directly on `System.IO.Compression` with a more ergonomic API and path-traversal protection.

Chuvadi.Sheets.Internal

Implementation. Not part of the public contract; types here may change between minor versions. Documented in this guide only where reading the implementation helps clarify the public API.

A typical short program using the library is on the order of five lines:

```
C#
using Chuvadi.Sheets.Excel;

var patients = new[] {
    new { Id = "P001", Name = "Anand Kumar", Dob = new DateTime(1962, 4, 12) },
    new { Id = "P002", Name = "Priya Sharma", Dob = new DateTime(1979, 11, 3) },
};

patients.ToXlsx("patients.xlsx");
```

The output is a single-sheet `xlsx` with three columns and two rows of data, opens correctly in Excel, LibreOffice, Google Sheets, and any tool that reads OOXML. There is no further setup, no template, no license check.

1.2 Why another `xlsx` library

The .NET ecosystem has several established `xlsx` libraries. They are mature and capable. Chuvadi.Sheets is not an attempt to replace them; it occupies a different point in the design space.

Concern	Established libraries	Chuvadi.Sheets
Dependencies	Variable; commonly 1–8 transitive packages	Zero NuGet dependencies. Pure BCL.
Licensing	Several require a paid license for commercial use beyond a row/file limit	MIT throughout
API surface	Large; some expose OOXML primitives directly	Small; OOXML is an implementation detail
Charts, pivots, drawings	Generally supported	Not supported by design
Workbook encryption	Supported in some, not others	Supported and Excel-verified
Streaming write/read	Often available as a separate API path	Streaming and random-access are both first-class
Native binaries	None typically required for write-only paths; some readers depend on native	None ever

If your application needs charts, pivot tables, embedded images, or workbook-level macros, look elsewhere. If your application needs to write reports, accept tabular data uploads, export query results, produce printable spreadsheets, or encrypt regulated content — and you want to do those things without taking on a dependency footprint or a license to vet — Chuvadi.Sheets is built for you.

WHEN TO CHOOSE THIS LIBRARY

The recurring use case is a backend service that produces `xlsx` files as outputs of business logic — invoices, audit logs, regulatory exports, patient registers, end-of-day reports — and a corresponding read path for files submitted by users. The encryption support is particularly relevant in healthcare, finance, and any domain with confidentiality requirements at the file level.

1.3 Feature overview

The library's capabilities, grouped by surface area:

Writing

- Streaming row-by-row writer with bounded memory (`XlsxWriter`, `SheetWriter`)
- Random-access workbook model where cells are addressed individually (`Workbook`, `Sheet`, `Cell`)
- One-liner exports from `IEnumerable<T>` and `DataTable` (`.ToXlsx()`)
- Multi-sheet exports composed from heterogeneous sources (`XlsxExport`)
- Async write paths throughout

Reading

- Streaming row-by-row reader with zero-allocation rows (`XlsxReader` , `SheetReader` , `RowReader`)
- Whole-workbook load into an editable model (`Workbook.Load`)
- One-liner reads into `DataTable` , `DataSet` , or `List<T>` (`.ReadXlsxSheet()` , `.ReadXlsx<T>()`)
- Auto-detection of date-typed cells from their style
- Forgiving parsing — minor format deviations don't fail the read

Cell styling

- Fonts (name, size, weight, italics, underline, color)
- Fills (background color)
- Borders (per-edge style and color)
- Number formats (numeric, date, currency, percentage, custom format codes)
- Alignment (horizontal, vertical, wrap-text)
- Built via a fluent `CellStyleBuilder` ; styles deduplicated automatically

Sheet features

- Column widths (per-column or per-range)
- Freeze panes (rows and columns)
- Merged cell ranges
- Autofilter
- Hyperlinks (URL, email, internal sheet links)
- Comments (notes attached to cells)
- Structured tables with built-in styles
- Defined names (named ranges and named scalars)

Data control

- Data validation — list (dropdown), whole number, decimal, date, text length
- Conditional formatting — color scales, data bars, cell-value comparisons
- Sheet protection with password (locked cells, restricted actions)
- Workbook structure protection (locked sheet tabs)
- Workbook open-password encryption (AES-256, OOXML Agile Encryption)

Zip utilities

- Streaming zip writer (`ZipWriter`)
- Streaming zip reader with safe extraction (`ZipReader`)
- One-liner directory zip / extract / listing (`ZipExtensions`)
- Path-traversal protection on both write and extract
- Async paths throughout

1.4 What it does not do

Software is partly defined by what it leaves out. Chuvadi.Sheets deliberately does not implement:

Charts

The OOXML chart schema is large and the rendering is largely Excel-specific. Tables and conditional formatting cover the most common reporting cases; for true chart output, a downstream chart-rendering library is the right tool.

Pivot tables

Same reasoning as charts: the schema is large and the consumer compatibility surface is narrow.

Embedded images and drawings

Image insertion via the DrawingML schema is technically straightforward but pulls in cross-references that complicate the streaming write path. Future versions may add this if there is real demand.

Rich text within cells

Excel allows multiple runs of differently-styled text within a single cell. Chuvadi.Sheets treats every cell value as a single uniformly-styled string. If you have a use case for rich text, a workaround is to put each run in its own cell and merge them.

Macros

The library writes `.xlsx` files, not `.xlsm`. There is no support for embedded VBA, signed macros, or any executable content. This is intentional: macro-enabled files are a security liability in many environments.

ZIP-level encryption

Older Office formats used zip-level password protection. Chuvadi.Sheets supports the modern OOXML Agile Encryption (AES-256), which is what Excel 2016+ produces by default.

Workbook-level password to modify

Excel has a "password to open" and a separate "password to modify". Chuvadi.Sheets supports the former; the latter is essentially advisory in Excel anyway and we have not implemented it.

If any of these features become essential to your use case, they may be candidates for future versions. The library is small enough that adding a feature touches a contained area of code; contributions on the GitHub repository are welcome.



2 • Installation

This chapter covers three installation paths in order of recommended adoption: local NuGet folder (for early integration and air-gapped builds), GitHub Packages (for private team distribution), and nuget.org (for public distribution).

The library targets **.NET 10** and can be retargeted to net8 or net9 by changing a single line in the `.csproj`. The dependencies it relies on — `System.IO.Compression`, `System.Xml`, `System.Security.Cryptography`, `System.Reflection` — are all part of the .NET base class library and available on every supported runtime.

2.1 From a local NuGet folder

The simplest distribution path: produce the `.nupkg` file on the build machine, drop it into a folder on the developer's machine, and add that folder as a NuGet source. The package then appears in Visual Studio's NuGet Package Manager exactly as if it came from nuget.org.

Producing the package

From the repository root:

Shell

```
dotnet pack src\Chuvadi.Sheets -c Release
```

This produces two files in `artifacts\bin\Chuvadi.Sheets\Release\`:

- `Chuvadi.Sheets.1.0.0.nupkg` — the package itself, around 340 KB
- `Chuvadi.Sheets.1.0.0.snupkg` — debug symbols for source-level debugging

Copy both files to a folder you can keep, for example `C:\NuGetLocal\`.

Configuring Visual Studio

In Visual Studio, open **Tools** → **NuGet Package Manager** → **Package Manager Settings** → **Package Sources**. Add a new source:

- **Name:** Chuvadi (or any name you like)
- **Source:** `C:\NuGetLocal` (the folder containing the `.nupkg`)

The new source appears in the Package Manager's source dropdown. Selecting it shows every `.nupkg` in the folder, including Chuvadi.Sheets.

Installing into a project

In any C# project that targets .NET 10 (or net8 / net9), open the NuGet Package Manager UI, select your local source, search for `Chuvadi.Sheets`, and click Install. Alternatively, from the Package Manager Console:

Shell

```
Install-Package Chuvadi.Sheets -Source Chuvadi
```

Or from the .NET CLI:

Shell

```
dotnet add package Chuvadi.Sheets --source C:\NuGetLocal
```

The package adds two entries to your `.csproj`:

XML

```
<ItemGroup>
  <PackageReference Include="Chuvadi.Sheets" Version="1.0.0" />
</ItemGroup>
```

FOR TEAM ADOPTION

A shared local folder on a build server or shared drive works the same way. Drop a new `.nupkg` version there, bump the package reference in consumers, and you have a working internal distribution channel with no infrastructure. This is also the right starting point for evaluating the library in a single project before formalising distribution.

2.2 From GitHub Packages

GitHub Packages provides a NuGet feed tied to a GitHub repository. Free for private use with reasonable limits. The publication command is a single line; the consumer-side configuration involves a personal access token (PAT) with `read:packages` scope.

Publishing

Shell

```
dotnet nuget push artifacts\bin\Chuvadi.Sheets\Release\Chuvadi.Sheets.1.0.0.nupkg `
  --api-key YOUR_GITHUB_PAT `
  --source https://nuget.pkg.github.com/aranshivab/index.json
```

Replace `aranshivab` with the owner of the repository, and `YOUR_GITHUB_PAT` with a personal access token that has the `write:packages` scope.

Consuming

On a consumer machine, add the GitHub Packages feed to NuGet's source list:

Shell

```
dotnet nuget add source https://nuget.pkg.github.com/arunshivab/index.json `
  --name github-chuvadi `
  --username YOUR_GITHUB_USERNAME `
  --password YOUR_GITHUB_PAT `
  --store-password-in-clear-text
```

Then install as normal:

Shell

```
dotnet add package Chuvadi.Sheets
```

PERSONAL ACCESS TOKENS

The `--store-password-in-clear-text` flag stores the PAT in the user's NuGet config file. For team or CI deployments, use environment variables or a secret manager instead, and rotate tokens regularly.

2.3 From nuget.org

Once the library has matured through internal use, public distribution on nuget.org makes it installable anywhere with a single command.

Create a free account at nuget.org, generate an API key, then push:

Shell

```
dotnet nuget push artifacts\bin\Chuvadi.Sheets\Release\Chuvadi.Sheets.1.0.0.nupkg `
  --api-key YOUR_NUGET_ORG_API_KEY `
  --source https://api.nuget.org/v3/index.json
```

Publication is asynchronous; the package usually becomes installable within minutes. Consumers install with no special configuration:

Shell

```
dotnet add package Chuvadi.Sheets
```

PUBLICATION IS DURABLE

nuget.org versions can be unlisted (hidden from search) but never truly deleted. Make sure each version you push is one you're willing to keep available. For experimentation, the local-folder path is appropriate.



3 · First example

The canonical "hello world" — read this even if you skim the rest of Part I. The program below produces a small xlsx file, then reads it back. It exercises four of the library's most-used APIs in fewer than thirty lines.

```

C#
using System;
using Chuvadi.Sheets.Excel;

public class Patient
{
    public string Id { get; set; } = "";
    public string Name { get; set; } = "";
    public DateTime Dob { get; set; }
    public bool Active { get; set; }
}

class Program
{
    static void Main()
    {
        // ---- Write -----
        var patients = new[]
        {
            new Patient { Id = "P001", Name = "Anand Kumar", Dob = new(1962, 4,
12), Active = true },
            new Patient { Id = "P002", Name = "Priya Sharma", Dob = new(1979, 11,
3), Active = true },
            new Patient { Id = "P003", Name = "Ravi Iyer", Dob = new(1985, 2,
27), Active = false },
        };

        patients.ToXlsx("patients.xlsx", cfg => cfg
            .SheetName("Patients")
            .HeaderStyle(new CellStyleBuilder()
                .Bold()
                .Background("#1A237E")
                .Foreground("#FFFFFF")
                .Build())
            .AutoFilter()
            .FreezeHeaderRow());

        Console.WriteLine($"Wrote patients.xlsx ({new Sys-
tem.IO.FileInfo("patients.xlsx").Length} bytes)");

        // ---- Read -----
        var loaded = "patients.xlsx".ReadXlsx<Patient>();

        foreach (var p in loaded)
            Console.WriteLine($"{p.Id} {p.Name, -20} {p.Dob:yyyy-MM-dd}
{(p.Active ? "active" : "inactive")}");
    }
}

```

Running this program produces an xlsx file with a styled header row (white text on deep indigo background), an autofilter on row 1, and frozen header so the column titles stay visible as the user scrolls. The read-back round-trips every value, including the booleans and the dates.

The same program rewritten with the streaming writer — for cases where the dataset is too large to materialize in memory — looks like this:

```
C#
using System;
using Chuvadi.Sheets.Excel;

using var writer = XlsxWriter.Create("patients.xlsx");
var sheet = writer.AddSheet("Patients")
    .FreezeRows(1)
    .SetColumnWidth(1, 8)    // Id
    .SetColumnWidth(2, 24)   // Name
    .SetColumnWidth(3, 14)   // Dob
    .SetColumnWidth(4, 10);  // Active

var headerStyle = new CellStyleBuilder()
    .Bold()
    .Background("#1A237E")
    .Foreground("#FFFFFF")
    .Build();

sheet.WriteRow(row => row
    .Cell("Id",      headerStyle)
    .Cell("Name",    headerStyle)
    .Cell("Dob",     headerStyle)
    .Cell("Active",  headerStyle));

foreach (var p in GetAllPatientsFromDatabase()) // can be millions of rows
{
    sheet.WriteRow(p.Id, p.Name, p.Dob, p.Active);
}

sheet.AutoFilter($"A1:D{sheet.RowsWritten}");
writer.Save();
```

Memory usage in this second program is bounded — a fixed-size buffer per sheet, regardless of how many rows pass through it. This is the right approach for any dataset whose size is unknown or potentially very large.

Both programs produce files that open identically in Excel, LibreOffice, and Google Sheets, with the same styling and behaviour. The difference is only in the API the application code uses; the on-disk format is the same.

WHICH TO USE

The fluent `.ToXlsx()` form is shorter and uses reflection on `T`'s public properties. The streaming `XlsxWriter` form is more verbose but lets you control everything — column widths, per-cell styles, formulas, hyperlinks, comments — without going through reflection. Chapter 10 discusses the trade-off in more depth.



4 • Library layout

The library is organised into three namespaces. This chapter is a brief tour; subsequent parts cover each namespace in depth.

Chuvadi.Sheets.Excel

The `xlsx` surface — the public API most applications interact with. Contains:

Writer types

`XlsxWriter`, `SheetWriter`, `RowBuilder`, `XlsxWriterOptions`

Reader types

`XlsxReader`, `SheetReader`, `RowReader`, `SheetInfo`, `XlsxReaderOptions`

Model types

`Workbook`, `Sheet`, `Cell`, `Range`, `Column`, `ColumnIndexer`, `DefinedNameCollection`

Styling types

`CellStyle`, `CellStyleBuilder`, `Border`, `BorderStyle`, `HorizontalAlign`, `VerticalAlign`, `TableStyle`

Feature types

`DataValidation`, `ConditionalFormat`, `ConditionalFormatRule`, `ComparisonOp`

Export types

`XlsxExtensions`, `XlsxReadExtensions`, `XlsxExport`, `XlsxExportConfig<T>`, `DataTableExportConfig`

Attribute types

`ColumnAttribute`, `ColumnIgnoreAttribute`, `ColumnStyleAttribute`

Protection types

`SheetProtectionOptions`, `EncryptionOptions`

Exception types

`XlsxFormatException`, `XlsxPasswordRequiredException`

Chuvadi.Sheets.Zip

The `zip` surface — a clean public API over `System.IO.Compression`:

Reader/writer

ZipReader, ZipWriter, ZipEntryInfo

Extensions

ZipExtensions (one-liners: ZipDirectory, ToZip, ExtractTo, ListZipEntries)

Exception

ZipFormatException

Chuvadi.Sheets.Internal

Implementation details. Not part of the public contract. The chapters of this guide refer to internal types only when reading their behaviour clarifies something about the public API. The main components, for orientation:

- `OoxmlPackage` — Open Packaging Conventions: parts, relationships, content types
- `StyleRegistry` — deduplicates styles, manages cellXf indices
- `SharedStringTable` — deduplicates strings, manages SST indices
- `WorkbookSerializer`, `WorkbookLoader` — bridges between the public `Workbook` model and the streaming reader/writer
- `ExportEngine` — backs the `.ToXlsx()` extensions
- `TypeAccessor` — reflection cache for property reads on T
- `Crypto.*` — agile encryption, password hashing, CFB container, data spaces records



Part II

Architecture



5 • Design principles

The library is built on four principles, each visible in the code:

Pure BCL, no exceptions

The `.csproj` declares no `PackageReference` elements other than internal links. Every operation — XML reading and writing, zip compression, AES encryption, PBKDF2 key derivation, HMAC integrity checks — uses types in `System.IO`, `System.Xml`, `System.Security.Cryptography`, `System.IO.Compression`, or `System.Reflection`. Nothing else. This decision shapes the rest of the design: features that would require a non-BCL dependency are simply not implemented.

The consequence is a library that is trivially auditable, has no version-skew risk against third-party packages, and can be deployed in air-gapped environments without procurement review.

Streaming is the default

Writing one row at a time produces the same on-disk file as building a model and writing it in one shot — but the streaming path uses bounded memory regardless of dataset size. The reading path is the same in reverse: one row at a time, with a reusable buffer that allocates only on the first row.

The random-access `Workbook` model exists alongside, for cases where streaming is awkward — building a workbook from heterogeneous sources, mutating individual cells in arbitrary order, reading a small file and editing it. Internally, `Workbook.SaveTo` drives an `XlsxWriter` session, so the random-access path is a thin layer over the streaming one.

Excel-correct, validated against external tools

Producing OOXML that the OOXML schema accepts is necessary but not sufficient — Excel is stricter than the schema. The encryption path in particular went through several rounds of validation against the real Microsoft Excel application and against the spec-strict `msoffcrypto-tool` Python library. Any file the library produces should open in Excel; any file Excel produces should be readable by the library.

This is not a guarantee for every corner of the OOXML spec — files using features the library does not implement (charts, pivots, drawings) will lose those features when loaded and re-saved through `Workbook.Load` → `SaveTo`. But the round-trip is lossless for the features the library does implement.

Predictable behaviour

The public API surface is deliberately small. Methods do one thing. Where there are multiple ways to express the same operation — a fluent extension method and a low-level writer call — they produce byte-identical output. There are no implicit conversions, no surprising mutations, and very few keyword arguments with non-obvious defaults.

The cost of this principle is occasional verbosity. The `RowBuilder` API requires you to declare every cell explicitly; there is no "auto-fill from this object" inside the streaming writer (only in the higher-level `.ToXlsx()` extension, which delegates to reflection). That verbosity is the price of being able to read a piece of code that uses the streaming writer and immediately know what it does.

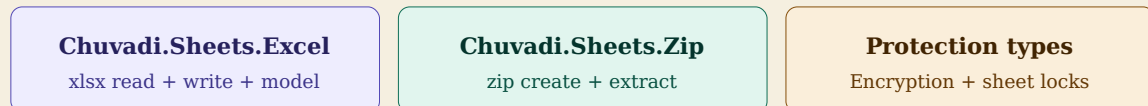


6 • Module map

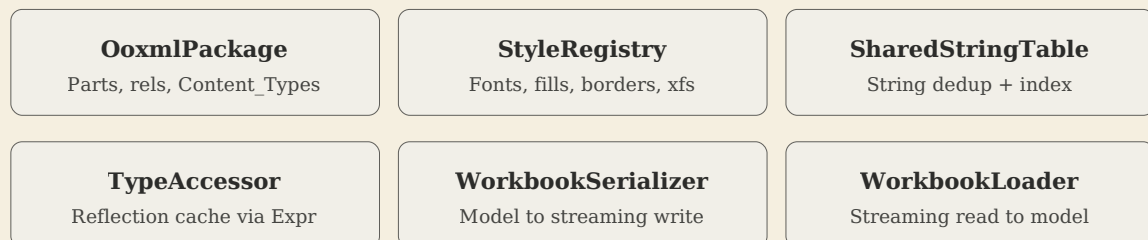
The diagram below shows the library's top-level structure: public API at the top, internal subsystems in the middle, the crypto stack as a self-contained corner, and the .NET base class library at the foundation.

Chuvadi.Sheets — v1.0 architecture

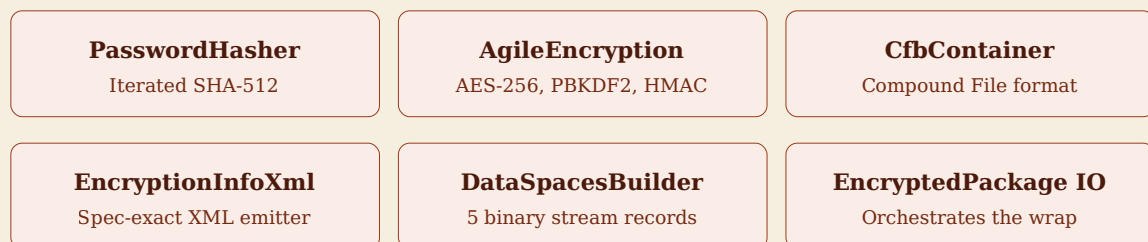
Public API



Internal subsystems



Crypto stack



Runtime foundation

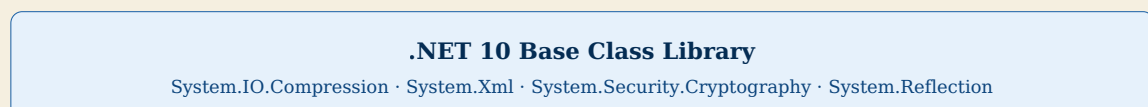


Figure 6.1 — Top-level module layout. Purple/teal/amber are public API surfaces, gray is internal infrastructure, coral is the encryption stack, blue is the BCL foundation.

Three lines tell the operating story:

- **Public types depend only on internal types.** Application code touches the top tier only.
- **Internal types depend only on the BCL and on other internal types within the same logical group.** The crypto stack is a self-contained subgraph; the OOXML packaging code does not know about encryption, and vice versa.
- **The BCL is the only external dependency.** Nothing in the library imports from outside `System.*`.



7 • Write pipeline

When application code calls `SaveTo`, `.ToXlsx()`, or finishes an `XlsxWriter` session, the bytes flow through a known sequence of stages:

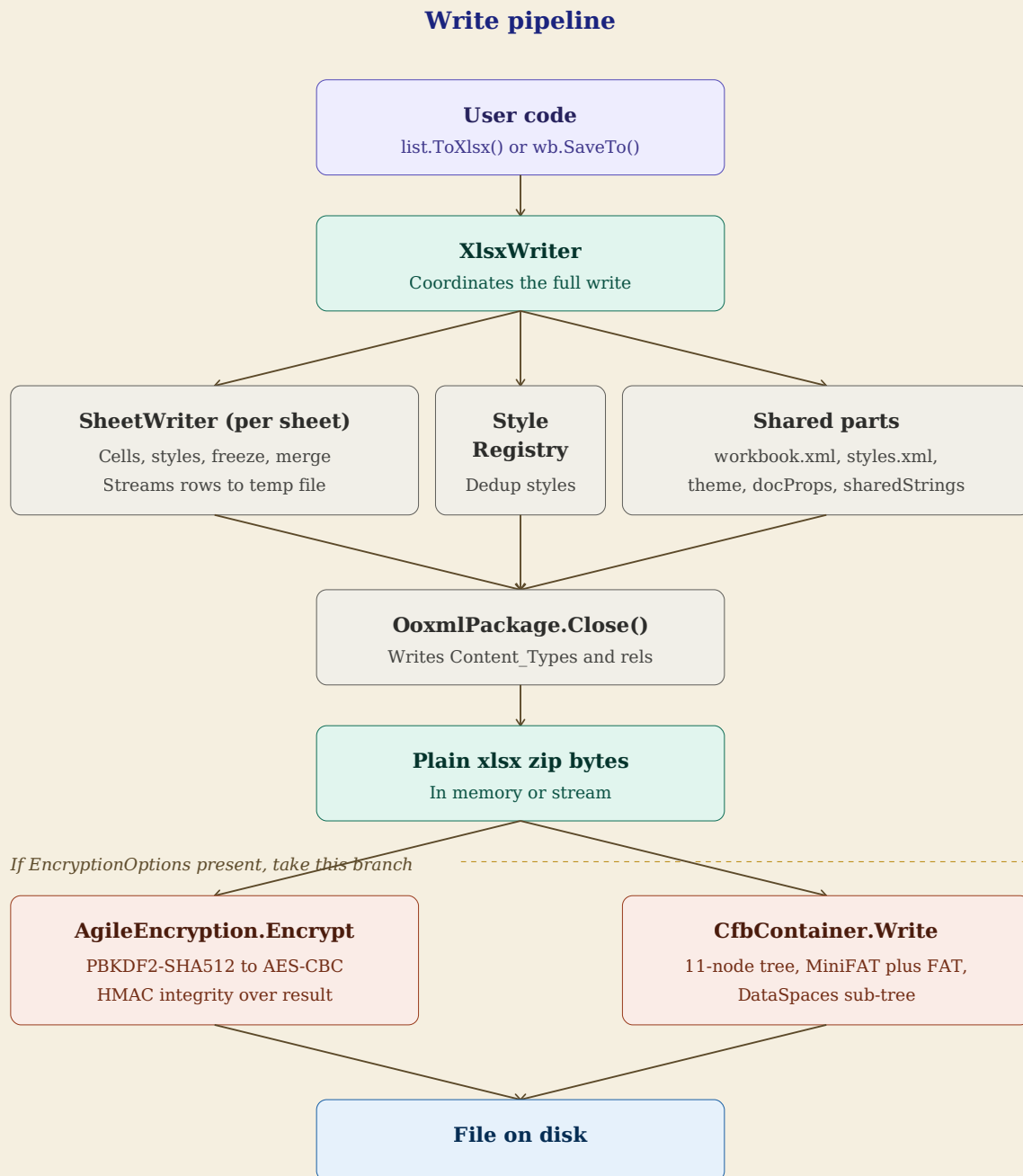


Figure 7.1 — Path from user code to a finished xlsx file. The lower half (below the gold dashed line) only runs when the call site supplies an `EncryptionOptions`.

The key observation: **plain xlsx zip bytes** is a real, well-defined intermediate. When encryption is requested, the library produces a complete plain xlsx in memory and then encrypts it. This separation keeps the encryption path testable and predictable, at the cost of double-buffering one file's worth of bytes during the encrypted save.

The temp files referred to in the SheetWriter box are per-sheet scratch files in `Path.GetTempPath()` (or wherever `XlsxWriterOptions.TempDirectory` points). They contain the partial sheet XML with shared-string placeholders. At the final assembly stage, those placeholders are resolved to real shared-string indices in a single byte-level pass.



8 · Read pipeline

Reads work in the inverse direction, with one additional consideration: the library has to detect whether the file is encrypted before deciding which pipeline to drive.

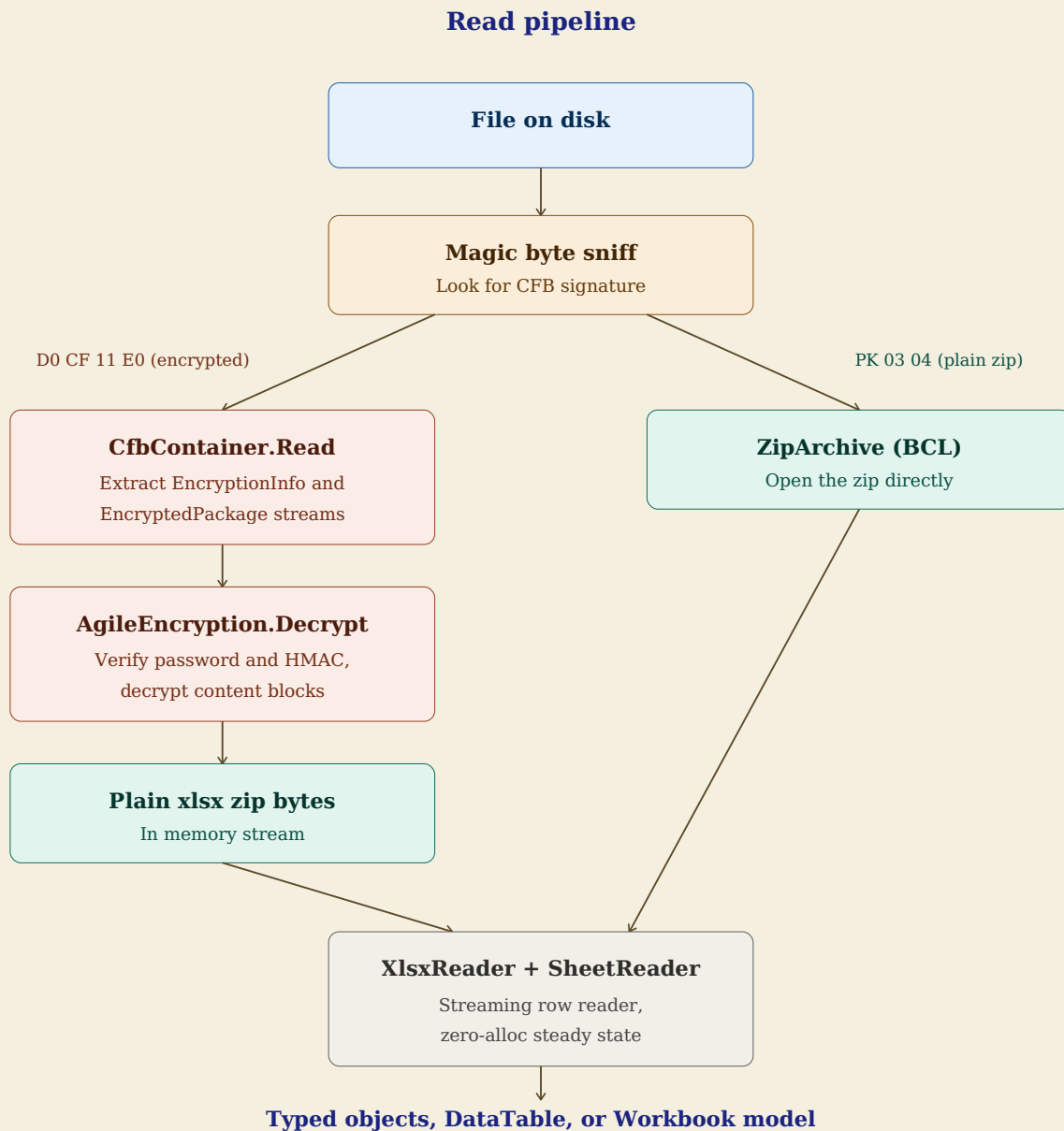


Figure 8.1 — Path from disk to in-memory representation. The two branches converge on a plain xlsx zip and from there share a single reading path.

The detection at the top of the diagram is a single read of the first eight bytes. If they match **D0 CF 11 E0 A1 B1 1A E1**, the file is a Compound File Binary container — that is, an encrypted xlsx. If they match **PK 03 04**, it is a plain zip. Anything else is rejected as not an xlsx.

After detection, the two pipelines do their own work to produce the same intermediate: a plain xlsx zip stream. From there, the streaming reader walks the standard OOXML parts and yields rows.



9 • Encrypted file format

This chapter is for readers who want to understand what an encrypted xlsx actually contains. Reading it is not necessary to use the encryption features — those are covered in Chapter 26 — but the structural picture helps when troubleshooting interoperability with other tools.

An encrypted xlsx is not just a password-protected zip. It is a **Compound File Binary** (CFB) container — Microsoft's older block-based file format, used as an envelope for the modern OOXML content. Inside the CFB envelope is a directory tree containing eleven entries: one large encrypted blob, one short metadata XML, and nine small binary records that tell Excel "this file uses Microsoft strong encryption on a stream named EncryptedPackage."

The tree's exact shape, matching the format Microsoft Excel itself produces:

```

Root Entry (storage)
├── EncryptedPackage (stream, regular FAT)
│   └── The actual encrypted xlsx – AES-CBC ciphertext
│       └── in 4096-byte segments, prefixed with the plaintext
│           └── length, integrity-protected by HMAC-SHA512.
├── \x06DataSpaces (storage – encryption metadata tree)
│   ├── Version (stream, MiniFAT)           76 bytes
│   ├── DataSpaceMap (stream, MiniFAT)      112 bytes
│   ├── DataSpaceInfo (storage)
│   │   └── StrongEncryptionDataSpace (stream, MiniFAT) 64 bytes
│   └── TransformInfo (storage)
│       └── StrongEncryptionTransform (storage)
│           └── \x06Primary (stream, MiniFAT)      200 bytes
└── EncryptionInfo (stream, MiniFAT)
    └── 8-byte v4.4 header + UTF-8 XML
        └── with salts, verifier blocks, spin count,
            └── and the encrypted content key.
```

Three things to notice:

1. **The plaintext xlsx is intact inside `EncryptedPackage`**. When Excel decrypts with the correct password, the result is a complete, valid xlsx zip file — exactly the same bytes that would have been written without encryption.
2. **The four small streams under `\x06DataSpaces` are byte-identical across files**. They are constants that declare the encryption transform; Chuvadi.Sheets produces them with the exact same bytes Excel does.
3. **`EncryptionInfo` carries everything needed to verify and decrypt**, except the password itself. The password is never stored; what is stored is a salted, iterated SHA-512 hash used to verify that an entered password is correct.

The cryptographic operations

Per the specification (MS-OFFCRYPTO §2.3.4), three operations happen at encryption time:

1. **Key derivation.** The user's password is concatenated with a 16-byte random salt and hashed with SHA-512. The hash is then re-hashed with a counter prefix, repeating for the configured spin count (default 100,000). This produces a 32-byte derived key.
2. **Content encryption.** A fresh random 32-byte key (the "content key") is generated. The xlsx zip bytes are encrypted with AES-256 in CBC mode using the content key, with a unique IV derived for each 4096-byte segment. The content key itself is then encrypted under the password-derived key.
3. **Integrity.** An HMAC-SHA512 is computed over the encrypted blob using a separate HMAC key. The HMAC key is itself encrypted under the content key. Excel verifies this HMAC before attempting decryption.

At decryption time, the same steps run in reverse: derive the key from the entered password, decrypt the content key, decrypt the HMAC key, verify the HMAC against the stored value, and only then decrypt the actual content.

SPECIFICATION REFERENCE

The complete format is described in [MS-OFFCRYPTO] sections 2.1.4 (DataSpaces records), 2.3.4 (Agile Encryption), 2.3.4.13 (AES with no padding), and 2.3.4.15 (encrypted key value). Chuvadi.Sheets implements the agile path (Version 4.4 with flags = 0x40) — the modern variant used by Excel 2016 and later.

What Chuvadi.Sheets does and does not implement

The library implements the agile encryption variant exclusively. The older RC4-based encryption schemes used by Office 95–2007 are deliberately not supported — they are cryptographically weak and writing them would be a security disservice.

The library does not implement the certificate-based key encryptor (where the content key is encrypted under a public key from an X.509 certificate). Only password-based encryption is supported. This matches the overwhelming majority of real-world use.



Part III

Writing xlsx files



10 · The two write APIs

Chuvadi.Sheets offers two ways to write an `xlsx` file. They produce identical output and can be mixed in the same program. The choice is about ergonomics and memory profile.

The streaming writer

The `XlsxWriter` + `SheetWriter` pair writes one row at a time directly to a temp file, which is then assembled into the final zip. Memory stays flat regardless of the row count. The API is procedural: you call methods in a specific order — set sheet shape (column widths, freeze panes) first, write rows, optionally add post-data features (autofilter, merge ranges), then dispose.

C#

```
using var writer = XlsxWriter.Create("output.xlsx");
var sheet = writer.AddSheet("Data");
sheet.FreezeRows(1);                                // sheet shape — must come first
sheet.SetColumnWidth(1, 24);

sheet.WriteRow("Id", "Name", "Score");               // header
sheet.WriteRow(1, "Alice", 87.5);
sheet.WriteRow(2, "Bob", 91.2);
sheet.WriteRow(3, "Carol", 78.0);

sheet.AutoFilter($"A1:C{sheet.RowsWritten}");
writer.Save();
```

The workbook model

The `Workbook` + `Sheet` + `Cell` tree lets you populate a sheet by addressing cells individually, in any order. Memory grows with the number of populated cells. The API is declarative: you build the whole model and then save it.

C#

```
var wb = new Workbook();
var sheet = wb.AddSheet("Data");
sheet.FreezeRows(1);
sheet.Columns[1].Width = 24;

sheet.Cell("A1").Value = "Id";
sheet.Cell("B1").Value = "Name";
sheet.Cell("C1").Value = "Score";
sheet.Cell("A2").Value = 1;
sheet.Cell("B2").Value = "Alice";
sheet.Cell("C2").Value = 87.5;
// ... etc.

wb.SaveTo("output.xlsx");
```

The ergonomic exports

A third path, layered on top of the streaming writer, is the `.ToXlsx()` extension method on `IEnumerable<T>` and `DataTable`. It uses reflection to map an object's public properties to columns, with attributes or a fluent config for customisation.

C#

```
records.ToXlsx("output.xlsx", cfg => cfg
    .SheetName("Data")
    .HeaderStyle(headerStyle)
    .AutoFilter()
    .FreezeHeaderRow());
```

This is shortest and works for the common case — one rectangular table, one sheet.

Choosing between them

If your situation is ...	Use
One rectangular table from an existing collection	<code>.ToXlsx()</code>
Multiple sheets from heterogeneous sources	<code>XlsxExport</code>
Building cells in arbitrary order, sparse layouts	<code>Workbook</code>
Dataset too large to materialise	<code>XlsxWriter</code>
Per-cell control: formulas, hyperlinks, comments	<code>XlsxWriter</code> via <code>RowBuilder</code>
Mutating a loaded file	<code>Workbook.Load</code>



11 · Workbook model

The `Workbook` class is the random-access entry point. A Workbook owns a list of `Sheet` instances, a `DefinedNameCollection` for workbook-level named ranges, and optional structure protection.

Creating a workbook

C#

```
using Chuvadi.Sheets.Excel;

var wb = new Workbook();
var data = wb.AddSheet("Data");
var summary = wb.AddSheet("Summary");
```

`AddSheet` appends to the tab order and returns the new `Sheet`. Sheet names are case-insensitively unique within a workbook; passing a duplicate name throws an `InvalidOperationException`.

The Sheet class

A sheet is a sparse collection of cells plus per-sheet metadata. Cells are stored in a `Dictionary<(int, int), Cell>` keyed by (row, column), so cells you never touch consume no memory.

Addressing cells

Two addressing modes, equivalent:

C#

```
sheet.Cell("A1").Value = "Header";
sheet.Cell(1, 1).Value = "Header";    // (row 1, column 1)
```

The A1 form parses standard Excel addresses up to column `XFD` (Excel's max). The integer form is more convenient when row and column indices are themselves variables.

Setting values

Cell content is exposed through two properties:

C#

```

sheet.Cell("A1").Value = 42;           // numeric
sheet.Cell("A2").Value = "Alice";      // string
sheet.Cell("A3").Value = DateTime.Today; // date
sheet.Cell("A4").Value = true;         // boolean
sheet.Cell("A5").Formula = "SUM(A1:A4)"; // formula (no '=' prefix)

```

Setting `Value` clears any previously-set `Formula`, and vice versa. The cell holds either a value or a formula, not both. Excel will compute the formula when the file is opened — Chuvadi.Sheets does not store cached values.

Cell styles

C#

```

var headerStyle = new CellStyleBuilder()
    .Bold()
    .Background("#1A237E")
    .Foreground("#FFFFFF")
    .HAlign(HorizontalAlign.Center)
    .Build();

sheet.Cell("A1").Style = headerStyle;
sheet.Cell("B1").Style = headerStyle;
sheet.Cell("C1").Style = headerStyle;

```

The same `CellStyle` instance can be reused across cells; the library deduplicates internally and emits each unique style once into `styles.xml`.

Sheet shape

C#

```

sheet.Columns[1].Width = 12;           // column A
sheet.Columns[2].Width = 24;           // column B
sheet.FreezeRows(1);                   // header row stays visible
sheet.FreezeColumns(0);                 // no frozen columns
sheet.AutoFilter("A1:E1000");           // filter dropdowns on row 1
sheet.MergeCells("A1:E1");              // span the title across columns A-E

```

Ranges

`Sheet.Range(address)` returns a `Range` struct over the given A1 range. The struct is small; its only operation is `.Merge()`, which is equivalent to calling `MergeCells(address)`.

C#

```

sheet.Range("A1:E1").Merge();           // merge title row
sheet.Range("A3:A10").Merge();          // merge a column range

```

Saving

Three forms of save:

```
C#
wb.SaveTo("output.xlsx");           // to a file
wb.SaveTo(someStream);             // to a stream
wb.SaveTo("secret.xlsx", new EncryptionOptions // encrypted
{
    Password = "MyP@ssw0rd!",
    SpinCount = 100_000,
});
```

The stream overload does not close the stream — the caller owns the stream's lifetime. The file overload creates or overwrites the target.

Loading

Symmetric to save:

```
C#
var loaded = Workbook.Load("input.xlsx");
var fromStream = Workbook.Load(someStream);
var encryptedLoaded = Workbook.Load("secret.xlsx", "MyP@ssw0rd!");
```

LOSSY LOAD

`Workbook.Load` preserves the features the model supports — cell values, formulas, number formats, column widths, freeze panes, merged ranges, defined names. It silently drops charts, pivot tables, embedded images, comments, custom table styles, and conditional formats on save. For read-only access without losing those features, use `XlsxReader` directly.

Workbook-scoped features

Defined names

Workbook-scoped names usable in formulas:

```
C#
wb.NamedRanges.Add("TaxRate", "0.18");
wb.NamedRanges.Add("DataRange", "Data!$A$2:$E$1000");

sheet.Cell("F2").Formula = "Subtotal * TaxRate"; // refers to TaxRate
```

Names must start with a letter or underscore and contain no spaces or special characters. The check is enforced at `Add` time; malformed names throw `ArgumentException`.

Workbook structure protection

C#

```
wb.Protect("structurepass", lockStructure: true, lockWindows: false);
```

This locks the sheet tabs — users cannot add, remove, rename, or reorder sheets without the password. The password is hashed (not stored in plaintext); see Chapter 25 for details.

SHEET PROTECTION IS SEPARATE

Workbook structure protection and per-sheet protection are independent. A workbook can have: - Both: tabs are locked AND specific sheets refuse cell edits. - Either alone. - Neither. Each sheet's own protection is set via `sheet.Protect(password, options)` — see Chapter 24.



12 · Streaming writer

The streaming writer is the API to use when datasets might be large, when memory is constrained, or when the write order naturally follows the row order (which is most reporting code). It is also the foundation that the workbook model and the ergonomic exports build on; understanding it pays off.

Construction

```
C#
using var writer = XlsxWriter.Create("output.xlsx");
// or:
using var writer = XlsxWriter.Create(stream);
// or with options:
using var writer = XlsxWriter.Create("output.xlsx", new XlsxWriterOptions
{
    UseInlineStrings = false,
    DefaultDateFormat = "dd-MMM-yyyy",
});
```

The `using` declaration matters: the writer holds temp file handles and a write buffer. Disposing without calling `Save` first cleans them up but does not produce a valid output file.

Adding sheets

```
C#
var dataSheet = writer.AddSheet("Data");
var summarySheet = writer.AddSheet("Summary");
```

Multiple sheets can be open at once. Each gets its own temp file; the final assembly merges them at `Save`.

Setting sheet shape (must come first)

Before writing the first row, set anything that affects the worksheet's structural elements: column widths and freeze panes.

C#

```
dataSheet
    .FreezeRows(1)
    .FreezeColumns(2)
    .SetColumnWidth(1, 12)           // column A: 12
    .SetColumnWidth(2, 24)           // column B: 24
    .SetColumnWidth(3, 10, 18);      // columns C-10: width 18 each
```

After the first row is written, these methods throw `InvalidOperationException`. The library can only emit them in the right place if they're set before the worksheet element opens.

Writing rows

Three forms of row write, in increasing flexibility:

WriteHeader — string header row

C#

```
sheet.WriteHeader("Id", "Name", "Date of birth", "Active");
```

Shorthand for "write a row of strings with default styling." Equivalent to `WriteRow("Id", "Name", "Date of birth", "Active")` but reads better as a header.

WriteRow(params object?[]) — values with runtime typing

C#

```
sheet.WriteRow(1, "Alice", new DateTime(1990, 5, 1), true);
sheet.WriteRow(2, "Bob", new DateTime(1985, 8, 12), false);
```

Each value is type-dispatched at runtime. The library recognises and correctly types: `string`, `bool`, `int`, `long`, `short`, `byte`, `uint`, `ulong`, `sbyte`, `ushort`, `float`, `double`, `decimal`, `DateTime`, `DateOnly`, `DateTimeOffset`, `TimeSpan`, and `null`. Other types are written via `ToString()`.

WriteRow(Action<RowBuilder>) — fluent per-cell control

C#

```
sheet.WriteRow(row => row
    .Cell("Alice")
    .Cell(42, importantStyle)
    .Formula("SUM(B1:B10)")
    .Cell(DateTime.Now)
    .Empty()
    .Hyperlink("mailto:alice@example.com", "alice@example.com")
    .Comment("On leave", "Mary", "Out until Monday"));
```

The RowBuilder API gives you a separate method for each kind of cell: `Cell` (auto-typed value, optional style), `Formula`, `Empty`, `Hyperlink`, and `Comment`. Each call appends one cell. The builder is the only way to write formulas, hyperlinks, and comments through the streaming API.

Async writes

Mirror methods exist for asynchronous I/O:

C#

```
await sheet.WriteHeaderAsync("Id", "Name");
await sheet.WriteRowAsync(1, "Alice");
await writer.SaveAsync(cancellationToken);
```

The underlying file stream is opened with `FileOptions.Asynchronous` so I/O is genuinely non-blocking.

Saving

C#

```
writer.Save();
// or:
await writer.SaveAsync(ct);
```

Save is mandatory. Disposing the writer without calling `Save` first cleans up temp files but produces no output. The reason is that the final assembly stage — resolving shared-string placeholders, writing `workbook.xml`, `[Content_Types].xml`, and relationships, and assembling the zip — only runs in `Save`.

After `Save` returns, the writer is closed for further mutation. Subsequent calls throw `InvalidOperationException`.

Sheet-level features set after rows

Some features depend on knowing the final row count and are therefore set after the rows are written, just before save:

C #

```
sheet.AutoFilter($"A1:D{sheet.RowsWritten}");  
sheet.MergeCells("A1:D1");  
sheet.AddDataValidation("E2:E1000", DataValidation.List("Active", "Inactive"));  
sheet.AddConditionalFormat("D2:D1000", ConditionalFormat.ColorScale.RedWhiteGreen);  
sheet.AddTable($"A1:D{sheet.RowsWritten}", "Patients", TableStyle.Medium2,  
    "Id", "Name", "Dob", "Active");  
sheet.Protect("editpass");
```

All of these are valid after `WriteRow` but before `writer.Save()`.



13 · Cells, values, types

This chapter is a reference for what happens when a value of a given .NET type lands in a cell. The behaviour is the same whether the value comes from `WriteRow`, the `RowBuilder.Cell` call, or a `Sheet.Cell(...).Value` assignment.

Strings

String cells are written either inline (each cell carries its own `<t>` element) or through the shared-strings table (each unique string appears once in `sharedStrings.xml` and cells reference it by index). The choice is controlled by `XlsxWriterOptions.UseInlineStrings`; default is `false` (shared strings).

Mode		File size		Memory	Save time
Shared strings (default)		Smaller	when strings repeat	Higher (the SST is built in memory)	Slower (post-processing pass to resolve placeholders)
Inline strings		Larger		Lower	Faster

For typical tabular data — sheets with many repeated values — shared strings produce noticeably smaller files. For data that is mostly unique strings (logs, free-text fields), inline mode is faster and the file-size penalty is minimal.

Numbers

All integer and floating-point types — `int`, `long`, `short`, `byte` and their unsigned/signed variants, plus `float`, `double`, `decimal` — are written as Excel numeric cells. They round-trip losslessly within the precision Excel itself supports (15 significant decimal digits for floating point).

Decimals are converted to `double` for storage, since the OOXML format does not distinguish. This is lossy only for values that exceed double precision (rare in practice).

Booleans

Written as Excel boolean cells (`t="b"`). The cell value is `"1"` or `"0"`; Excel displays it as `TRUE` or `FALSE`.

Dates and times

Excel stores dates as numbers — the count of days since 1899-12-30, with the time of day as the fractional part. Chuvadi.Sheets handles the conversion for you:

.NET type	Stored as	Default format
<code>DateTime</code> (date only)	OADate (integer)	<code>yyyy-mm-dd</code>
<code>DateTime</code> (with time)	OADate (fractional)	<code>yyyy-mm-dd hh:mm:ss</code>
<code>DateOnly</code>	OADate (integer)	<code>yyyy-mm-dd</code>
<code>DateTimeOffset</code>	OADate (fractional, local time)	Auto-detected
<code>TimeSpan</code>	Days as a decimal	<code>[h]:mm:ss</code>

The default formats come from `XlsxWriterOptions` and can be customized:

```
C#
var writer = XlsxWriter.Create("output.xlsx", new XlsxWriterOptions
{
    DefaultDateFormat      = "dd MMM yyyy",
    DefaultDateTimeFormat  = "yyyy-MM-dd HH:mm",
    DefaultTimeSpanFormat  = "h:mm:ss",
});
```

Per-cell overrides win — if a cell has a `Style` with a `NumberFormat` set, that format takes precedence over the default.

THE 1900 LEAP YEAR BUG

Excel treats 1900 as a leap year even though it isn't. This dates back to a Lotus 1-2-3 compatibility decision in 1985. `DateTime.ToOADate()` handles it correctly. Just be aware that dates between 1900-01-01 and 1900-02-28 may appear shifted by one day when round-tripped through some non-Excel xlsx libraries.

Formulas

Formulas are written as the expression text without the leading `=` sign. Excel computes the value when the file opens:

C#

```
sheet.WriteRow(row => row
    .Cell("Total")
    .Formula("SUM(B2:B100)")
    .Formula("AVERAGE(B2:B100)")
    .Formula("IF(D2 > 1000, \"High\", \"Normal\")"));
```

Chuvadi.Sheets does not validate or evaluate formulas. If the formula text is malformed, Excel will show `#NAME?` or similar when the file opens. If the formula references defined names, those names must be declared via `DefinedNames.Add` or via `XlsxWriter.SetDefinedNames` at the writer level.

Null and missing values

`null` values produce an empty cell. The column index advances; the cell exists in the XML structure but has no content. For the streaming writer, `RowBuilder.Empty()` is equivalent.



14 · Cell styling

Cell appearance is controlled by a `CellStyle` — an immutable value type. The recommended way to build one is the fluent `CellStyleBuilder`.

The `CellStyle` record

`CellStyle` is a `readonly record struct` with fields for fonts, fills, borders, number format, and alignment. As a value type, it has structural equality — two `CellStyles` with the same fields are equal and produce the same hash. The library uses this for deduplication: passing the same style to multiple cells creates one entry in `styles.xml`, not many.

The fluent builder

C#

```
var style = new CellStyleBuilder()
    .Font("Calibri", 11)
    .Bold()
    .Italic(false)
    .Foreground("#FFFFFF")
    .Background("#1A237E")
    .HAlign(HorizontalAlign.Center)
    .VAlign(VerticalAlign.Center)
    .BorderAll(BorderStyle.Thin, "#666666")
    .Format("#,##0.00")
    .WrapText()
    .Build();
```

All builder methods return the builder, so chaining works. `Build()` materialises the immutable `CellStyle`. The builder is reusable — you can build, mutate, and build again.

Colors

Colors are 6-digit hex RGB strings. The leading `#` is optional:

C#

```
.Foreground("#FF0000")    // red
.Foreground("FF0000")    // same
.Background("#1A237E")    // deep indigo
```

The builder normalizes the input to the no-hash uppercase form Excel writes internally (`FF0000`). Malformed colors throw `ArgumentException` at build time, surfacing the problem at the call site rather than producing a broken file.

Borders

A `Border` is a (style, color) pair. Borders are set per-edge, with a convenience for all-edges-the-same:

C#

```
// All four edges, same style and color
.BorderAll(BorderStyle.Thin, "#666666")

// Different per edge
.BorderTop(BorderStyle.Thick, "#1A237E")
.BorderBottom(BorderStyle.Thin, "#666666")
.BorderLeft(BorderStyle.None, "")
.BorderRight(BorderStyle.None, "")
```

Available `BorderStyle` values: `None`, `Thin`, `Medium`, `Dashed`, `Dotted`, `Thick`, `Double`, `Hair`.

Number formats

Number format codes follow Excel's syntax. The library does not validate them — what you pass is what Excel sees. Some commonly useful codes:

Code	Effect	Example
<code>0</code>	Integer	42
<code>0.00</code>	Two decimal places	42.00
<code>#,##0</code>	Thousands separator, integer	1,234,567
<code>#,##0.00</code>	Thousands, two decimals	1,234,567.89
<code>0.00%</code>	Percentage	15.00%
<code>0.00E+00</code>	Scientific notation	1.23E+06
<code>"₹"#,##0.00</code>	Currency with rupee prefix	₹1,234.56
<code>yyyy-mm-dd</code>	ISO date	2026-05-26
<code>dd-mmm-yy</code>	Short month date	26-May-26
<code>hh:mm:ss</code>	Time	14:35:22
<code>[h]:mm:ss</code>	Hours not truncated at 24	72:00:00
<code>@</code>	Force text	"42" (not 42)

Alignment

Horizontal alignment values (`HorizontalAlign` enum): `General`, `Left`, `Center`, `Right`, `Fill`, `Justify`, `CenterContinuous`, `Distributed`. `General` is Excel's default — text left, numbers right, booleans centered.

Vertical alignment values (`VerticalAlign` enum): `Top`, `Center`, `Bottom`, `Justify`, `Distributed`.

`wrapText(true)` makes the cell wrap long content across multiple visual lines. Excel also adjusts row height to fit when this is on.

Style deduplication

The library deduplicates styles via the value-type equality of `CellStyle`. The implication for user code: **build the style once, reuse the instance**. All of these produce the same single entry in styles.xml:

C#

```
// Reused – efficient
var headerStyle = new CellStyleBuilder().Bold().Background("#1A237E").Build();
for (int i = 1; i <= 10; i++)
    sheet.Cell(1, i).Style = headerStyle;

// Built fresh each time – also efficient (deduplication catches it)
for (int i = 1; i <= 10; i++)
    sheet.Cell(1, i).Style = new CellStyleBuild-
er().Bold().Background("#1A237E").Build();
```

The second form does more work at build time (allocating ten builders) but produces the same output file with one style entry.



15 · Ergonomic exports

For the common case of "rectangular data into a spreadsheet," Chuvadi.Sheets provides extension methods that handle reflection, column mapping, and formatting in one call.

Export an IEnumerable<T>

The simplest form:

```
C#
using Chuvadi.Sheets.Excel;

public class Patient
{
    public string Id { get; set; } = "";
    public string Name { get; set; } = "";
    public DateTime Dob { get; set; }
    public bool Active { get; set; }
}

var patients = LoadPatientsFromDatabase();
patients.ToXlsx("patients.xlsx");
```

This produces a single-sheet xlsx with one column per public property of `Patient`, in declaration order, with the property name as the header.

Customising the export

A configuration lambda lets you customize everything: sheet name, header styling, autofilter, freeze panes, table conversion, per-column overrides, and column exclusions.

C#

```
patients.ToXlsx("patients.xlsx", cfg => cfg
    .SheetName("Patient register")
    .HeaderStyle(new CellStyleBuilder()
        .Bold()
        .Background("#1A237E")
        .Foreground("#FFFFFF")
        .HAlign(HorizontalAlign.Center)
        .Build())
    .AutoFilter()
    .FreezeHeaderRow()
    .Column(p => p.Id, header: "Patient ID", width: 14)
    .Column(p => p.Dob, header: "Date of birth", width: 14, format: "yyyy-mm-dd")
    .Column(p => p.Active, header: "Status", width: 10)
    .Ignore(p => p.InternalNotes));
```

Notable points:

- `.Column(p => p.PropertyName, ...)` uses an expression to identify the property, so it's refactor-safe — renaming a property updates the export.
- Configuration overrides any attributes on the class.
- Columns appear in the order declared on the class unless an `[Order]` attribute or `.Column(..., order: ...)` reorders them.

Attributes on the data class

An alternative to the fluent config — useful when the same class is exported from many places. The three attributes:

C#

```
public class Patient
{
    [Column("Patient ID", Order = 1, Width = 14)]
    public string Id { get; set; } = "";

    [Column("Patient name", Order = 2, Width = 24)]
    public string Name { get; set; } = "";

    [Column("Date of birth", Order = 3, Width = 14, Format = "yyyy-mm-dd")]
    public DateTime Dob { get; set; }

    [Column("Status", Order = 4, Width = 10)]
    [ColumnStyle(Bold = true, Background = "#F0E9D0")]
    public bool Active { get; set; }

    [ColumnIgnore]
    public string InternalNotes { get; set; } = "";
}
```

Wrap as a table

C #

```
patients.ToXlsx("patients.xlsx", cfg => cfg
    .AsTable("Patients", TableStyle.Medium2)
    .FreezeHeaderRow());
```

The output is wrapped in an Excel structured table with filter dropdowns, banded rows in the indigo Medium2 style, and a structured-reference name `Patients` that can be used in formulas: `=COUNTA(Patients[Name])`.

DataTable export

Symmetric API for `System.Data.DataTable`:

C #

```
DataTable dt = QueryDatabase("SELECT * FROM patients");
dt.ToXlsx("patients.xlsx", cfg => cfg
    .SheetName(dt.TableName ?? "Data")
    .AutoFilter()
    .Column("Dob", format: "yyyy-mm-dd")
    .Column("Salary", format: "#,##0.00"));
```

Multi-sheet export

For workbooks with multiple sheets sourced from different collections:

C #

```
new XlsxExport("report.xlsx")
    .AddSheet("Patients", patients, cfg => cfg
        .HeaderStyle(headerStyle)
        .AutoFilter()
        .FreezeHeaderRow())
    .AddSheet("Visits", visitsDataTable)
    .AddSheet("Billing", billing, cfg => cfg
        .AsTable("Billing", TableStyle.Medium4)
        .Column(b => b.Amount, format: "₹#,##0.00"))
    .Save();
```

Each `AddSheet` takes its own configuration lambda. Sheets appear in the order they're added.



16 · Sheet features

Beyond cells and styling, sheets carry several features that affect layout and behaviour in Excel.

Column widths

```
C#  
// Streaming writer  
sheet.SetColumnWidth(1, 12);           // column A  
sheet.SetColumnWidth(2, 5, 18);        // columns B-E: width 18 each  
  
// Workbook model  
sheet.Columns[1].Width = 12;
```

Width units are Excel's column-width units: roughly the number of digits of the default font that fit in the column. A width of 12 holds about twelve characters of Calibri 11.

Freeze panes

```
C#  
sheet.FreezeRows(1);                   // header row stays visible  
sheet.FreezeColumns(2);                 // first two columns stay visible  
sheet.FreezeRows(1).FreezeColumns(2); // both
```

Pass 0 to clear. Freezing must be set before the first row is written (streaming writer) or before save (workbook model).

Merged cells

```
C#  
sheet.MergeCells("A1:E1");             // span title across columns  
sheet.MergeCells("A2:A10");            // span a column down
```

The displayed value comes from the top-left cell of the range. Excel allows nested merge ranges but Chuvadi.Sheets does not validate against overlaps — passing overlapping ranges produces files Excel will warn about on open.

Autofilter

C #

```
sheet.AutoFilter("A1:D1000");
```

Adds filter dropdowns to the top row of the range. The range typically includes the header plus all data rows.

Hyperlinks

Through the `RowBuilder` API in the streaming writer:

C #

```
sheet.WriteRow(row => row
    .Cell("Visit Dr. Khan's page")
    .Hyperlink("https://example.com/drkhan", "View profile")
    .Hyperlink("mailto:khan@example.com", "Email Dr. Khan", tooltip: "Send email"));
```

The cell's display text shows in normal cell rendering; the URL is stored as a sheet relationship. Excel renders hyperlinked cells in the standard blue underline unless you pass a custom style.

Comments

C #

```
sheet.WriteRow(row => row
    .Cell("Patient 47")
    .Comment("On leave", "Mary", "Out until 2026-06-01")
    .Cell(42));
```

The cell value is written normally; Excel displays a small red triangle in the corner indicating a note is present. Hovering reveals the comment text with the author name.

Structured tables

C #

```
sheet.AddTable($"A1:D{sheet.RowsWritten}", "Patients", TableStyle.Medium2,
    "Id", "Name", "Dob", "Active");
```

Wraps the range in a structured table — filter dropdowns, banded row colors, structured references in formulas. The header strings passed must match what's actually in row 1.

Available styles: `None`, `Light1`–`Light21`, `Medium1`–`Medium28`, `Dark1`–`Dark11`. The `Medium2` style (blue) is Excel's default. For dim/subtle table styling, try `Light8`. For high contrast, try `Dark2` or `Dark9`.



17 · Data validation

Data validation rules restrict what users can type into cells when the file is open in Excel. Several built-in rule types are available via the `DataValidation` static factory.

Dropdown list

C#

```
sheet.AddDataValidation("E2:E1000",  
    DataValidation.List("Pending", "Active", "Closed", "Archived"));
```

The cell displays a dropdown arrow and refuses values not in the list. The inline list form is limited to a total of 255 characters; for longer lists, use `ListFromRange` to point at a named range:

C#

```
// Define a name pointing at the list  
wb.NamedRanges.Add("StatusList", "Lookups!$A$1:$A$50");  
  
// Reference it in the validation  
sheet.AddDataValidation("E2:E1000", DataValidation.ListFromRange("StatusList"));
```

Numeric ranges

C#

```
// Whole number between 0 and 120  
sheet.AddDataValidation("F2:F1000",  
    DataValidation.WholeNumber(min: 0, max: 120));  
  
// Decimal between 0.0 and 100.0  
sheet.AddDataValidation("G2:G1000",  
    DataValidation.DecimalNumber(min: 0.0, max: 100.0));  
  
// Whole number at most 1000 (no minimum)  
sheet.AddDataValidation("H2:H1000",  
    DataValidation.WholeNumber(max: 1000));
```

Date ranges

C#

```
sheet.AddDataValidation("I2:I1000",  
    DataValidation.Date(  
        min: new DateTime(2020, 1, 1),  
        max: DateTime.Today));
```

Text length

C#

```
// 1 to 50 characters  
sheet.AddDataValidation("B2:B1000",  
    DataValidation.TextLength(min: 1, max: 50));
```

Customising the validation message

Every rule exposes optional properties for the error dialog and the info popup:

C#

```
var rule = DataValidation.WholeNumber(min: 0, max: 120);  
rule.ErrorTitle    = "Invalid age";  
rule.ErrorMessage  = "Please enter an age between 0 and 120.";  
rule.PromptTitle   = "Age";  
rule.PromptMessage = "Enter the patient's age in years.";  
  
sheet.AddDataValidation("F2:F1000", rule);
```

`AllowBlank` (default true) controls whether empty cells pass the rule.

VALIDATION IS ADVISORY

Data validation rules apply in Excel's UI when users type values. They do *not* apply when values are inserted programmatically (via Chuvadi.Sheets or any other library). The library will happily write a value of 999 into a cell with a 0–120 validation rule, and the file will open fine — Excel will only flag the cell if the user later edits it.



18 · Conditional formatting

Conditional formatting rules change a cell's appearance based on its value, applied at render time by Excel.

Color scales

A color scale paints cells along a gradient based on their numeric value:

C #

```
sheet.AddConditionalFormat("B2:B100", ConditionalFormat.ColorScale.RedWhiteGreen);
sheet.AddConditionalFormat("C2:C100", ConditionalFormat.ColorScale.GreenWhiteRed);
sheet.AddConditionalFormat("D2:D100", ConditionalFormat.ColorScale.BlueWhiteRed);

// Custom three-color scale
sheet.AddConditionalFormat("E2:E100",
    ConditionalFormat.ColorScale.Custom("#FF0000", "#FFFF00", "#00FF00"));
```

Data bars

Data bars overlay a horizontal bar in the cell, scaled to the value:

C #

```
sheet.AddConditionalFormat("B2:B100", ConditionalFormat.DataBar.Blue);
sheet.AddConditionalFormat("C2:C100", ConditionalFormat.DataBar.Green);
sheet.AddConditionalFormat("D2:D100", ConditionalFormat.DataBar.Red);

// Custom color
sheet.AddConditionalFormat("E2:E100",
    ConditionalFormat.DataBar.Custom("#8B0000"));
```

Cell-value comparisons

Apply a style to cells matching a comparison:

C#

```
// Cells > 100 in bold red
sheet.AddConditionalFormat("B2:B100",
    ConditionalFormat.CellIs(ComparisonOp.GreaterThan, 100,
        new CellStyleBuilder().Bold().Foreground("#C62828").Build()));

// Cells between 50 and 100 in yellow
sheet.AddConditionalFormat("C2:C100",
    ConditionalFormat.Between(50, 100,
        new CellStyleBuilder().Background("#FFEB3B").Build()));
```

The full set of comparison operators in `ComparisonOp`: `Equal`, `NotEqual`, `GreaterThan`, `GreaterThanOrEqual`, `LessThan`, `LessThanOrEqual`, `Between`, `NotBetween`.



19 • Defined names

Defined names are workbook-scoped aliases for ranges, formulas, or scalar values. They make formulas more readable and centralise constants.

Through the Workbook model

C#

```
var wb = new Workbook();

wb.DefinedNames.Add("TaxRate", "0.18");
wb.DefinedNames.Add("Currency", "\\\"₹\\\"");
wb.DefinedNames.Add("DataRange", "Data!$A$2:$E$1000");

var sheet = wb.AddSheet("Invoice");
sheet.Cell("E2").Formula = "D2 * TaxRate";
sheet.Cell("F2").Formula = "TEXT(E2, Currency & \\\"#,##0.00\\\")";
```

Through the streaming writer

C#

```
var writer = XlsxWriter.Create("output.xlsx");
writer.SetDefinedNames(new Dictionary<string, string>
{
    ["TaxRate"] = "0.18",
    ["DataRange"] = "Data!$A$2:$E$1000",
});
```

Naming rules

Excel's rules for defined names, validated at `Add` time:

- Must start with a letter or underscore
- Cannot contain spaces or punctuation other than `_` and `.`
- Cannot conflict with cell addresses (e.g. `A1`, `R1C1`)
- Case-insensitive — `TaxRate` and `taxrate` refer to the same name

Malformed names throw `ArgumentException` from `Add`.



Part IV

Reading xlsx files



20 · Streaming reader

The `XlsxReader` opens an `xlsx` file and exposes its sheets as streaming row enumerators. Memory stays bounded — the reader holds the shared strings table and styles table in memory (typically small), and one row's worth of cell values in a reusable buffer.

Opening a file

```
C#
using Chuvadi.Sheets.Excel;

using var reader = XlsxReader.Open("patients.xlsx");

foreach (var sheetInfo in reader.Sheets)
    Console.WriteLine($"Sheet: {sheetInfo.Name}");
```

The reader is `IDisposable` — wrap it in `using` to ensure the file handle is released. `Open` takes a file path or an arbitrary `Stream`; the stream variant does not close the stream on dispose.

Reader options

```
C#
var reader = XlsxReader.Open("patients.xlsx", new XlsxReaderOptions
{
    TreatFirstRowAsHeaders = true,    // default
    AutoDetectDates        = true,    // default
    StrictXml               = false,   // default
});
```

TreatFirstRowAsHeaders

When true (default), the first row of each sheet is consumed to build a name → column-index map. Rows yielded by the enumerator start from row 2. Disable this when reading sheets that have no header row.

AutoDetectDates

When true (default), numeric cells with a date-like number format are returned as `DateTime` rather than `double`. The detection examines the cell's style; only styles with a date format code produce `DateTime` results.

StrictXml

When true, malformed XML inside any reachable sheet throws `XlsxFormatException`. When false (default), corrupt sheets are silently dropped from the read. Structural problems — broken zip, missing required parts — always throw regardless of this flag.

Accessing sheets

```
C#
// By name
var dataSheet = reader.Sheet("Data");

// By 1-based tab order
var firstSheet = reader.Sheet(1);
```

Sheet names are matched case-insensitively. Looking up a missing name throws `KeyNotFoundException`; an out-of-range index throws `ArgumentOutOfRangeException`.

Enumerating rows

```
C#
foreach (var row in dataSheet.Rows)
{
    string id    = row.GetString("Id");
    string name  = row.GetString("Name");
    DateTime dob = row.GetDateTime("Dob");
    bool active  = row.GetBool("Active");

    // process this row
}
```

The enumerator opens the sheet's XML stream on iteration start and disposes it when iteration ends or breaks. Multiple passes over the same sheet are allowed — each `foreach` opens a fresh enumerator.

The RowReader struct

`RowReader` is a `readonly struct` over the row buffer. The buffer is reused across iterations, which is what makes the steady-state allocation profile zero. The implication is critical:

DO NOT RETAIN ROWREADER INSTANCES ACROSS ITERATIONS

The struct's underlying buffer is reused. If you capture a `RowReader` and read it after the next `MoveNext`, you will see the next row's data, not the original. To use a row's values later, copy them out at iteration time.

The correct pattern:

```
C#
// CORRECT – copy out specific values
var copied = new List<(string Id, string Name)>();
foreach (var row in sheet.Rows)
{
    copied.Add((row.GetString("Id"), row.GetString("Name")));
}

// WRONG – RowReader will reference stale data
var collected = new List<RowReader>();
foreach (var row in sheet.Rows)
{
    collected.Add(row);    // by the next iteration, this row's data is overwritten
}
```

Cell access modes

`RowReader` supports four access modes:

C#

```
// Raw object access
object? rawById    = row[0];           // by 0-based column index
object? rawByName  = row["Id"];        // by header name

// Typed accessors – throw on missing/wrong type
string id          = row.GetString("Id");
int    count       = row.GetInt32("Count");
long   big         = row.GetInt64("Bigger");
double rate        = row.GetDouble("Rate");
decimal money      = row.GetDecimal("Amount");
bool   active      = row.GetBool("Active");
DateTime when      = row.GetDateTime("Date");

// Generic typed accessor
int v1 = row.Get<int>("Count");
T v2   = row.Get<T>(columnIndex);

// TryGet pattern – returns false instead of throwing
if (row.TryGet<int>("Count", out var c)) { /* ... */ }
if (row.TryGet<string>("Name", out var n)) { /* ... */ }
```

The typed accessors return `InvalidOperationException` for missing cells and `InvalidCastException` for cells of the wrong type. `TryGet` returns false in both cases without throwing.

The HeaderIndex

When `TreatFirstRowAsHeaders` is on, the sheet builds a case-insensitive map of header text to 0-based column index. The map is exposed as `sheet.HeaderIndex`, useful when you need to know what columns exist:

C#

```
foreach (var kv in sheet.HeaderIndex)
    Console.WriteLine($"{kv.Key} → column {kv.Value}");

// Check whether an optional column is present
if (sheet.HeaderIndex.ContainsKey("Phone"))
{
    // ...
}
```



21 · Loading a workbook

When you want to mutate an existing xlsx file — change some values, save it back — the streaming reader is the wrong tool. Use `Workbook.Load` instead.

C#

```
// Load a plain xlsx
var wb = Workbook.Load("input.xlsx");

// Mutate it
wb["Patients"].Cell("A2").Value = "Updated Name";
wb.AddSheet("Audit").Cell("A1").Value = "Loaded and edited";

// Save it back
wb.SaveTo("output.xlsx");
```

The load is best-effort: it preserves the data the workbook model supports and silently discards features the model does not represent.

What round-trips losslessly

- Cell values (strings, numbers, booleans, dates)
- Formulas (formula text is preserved; cached values are not stored)
- Sheet names and tab order
- Column widths
- Freeze panes
- Merged cell ranges
- Defined names
- Number formats applied to cells

What is dropped

- Charts
- Pivot tables
- Embedded images and drawings
- Comments / notes
- Conditional formatting
- Custom table styles
- Data validation rules
- Sheet and workbook protection

Loading encrypted files

The same `Workbook.Load` overload handles encrypted files transparently — pass the password as the second argument:

C#

```
var wb = Workbook.Load("encrypted.xlsx", "MyP@ssw0rd!");
```

If the file is encrypted and the password is wrong, `XlsxPasswordRequiredException` is thrown. If the file is encrypted and no password is supplied (one-arg overload), the same exception is thrown. If the file is not encrypted, the password is ignored.

Detection happens by sniffing the first eight bytes of the file: `D0 CF 11 E0 A1 B1 1A E1` indicates a CFB-wrapped encrypted package; `PK 03 04` indicates a plain zip.



22 · Ergonomic reads

The read equivalents of `.ToXlsx()`: one-liner methods for reading into `DataTable`, `DataSet`, or a strongly-typed `List<T>`.

Read into a DataSet

```
C#
using Chuvadi.Sheets.Excel;

DataSet ds = "report.xlsx".ReadXlsx();

foreach (DataTable table in ds.Tables)
{
    Console.WriteLine($"Sheet: {table.TableName} ({table.Rows.Count} rows)");
}
```

Every sheet becomes a `DataTable`. Column names come from the header row; values are stored as `object` (boxed primitives).

Read a single sheet into a DataTable

```
C#
DataTable patients = "patients.xlsx".ReadXlsxSheet("Patients");
DataTable firstSheet = "report.xlsx".ReadXlsxSheet(); // first sheet by default
```

Read into a typed list

```
C#
public class Patient
{
    public string Id { get; set; } = "";
    public string Name { get; set; } = "";
    public DateTime Dob { get; set; }
    public bool Active { get; set; }
}

List<Patient> patients = "patients.xlsx".ReadXlsx<Patient>();
List<Patient> fromNamedSheet = "report.xlsx".ReadXlsx<Patient>("Patients");
```

The mapping rules:

- Columns are matched to properties by header name (case-insensitive).
- Properties with `[Column("Custom header")]` use the custom name for matching.
- Properties with `[ColumnIgnore]` are skipped.
- Missing columns leave the property at its default value.
- Type conversion uses `Convert.ChangeType` for compatible types, with a special case for `DateTime` from a numeric serial.
- Conversion failures are silent — the property stays at its default value.
- Read-only properties (no setter) are skipped.

SILENT FAILURES ARE BY DESIGN

The ergonomic reader is forgiving. If a column is missing, mistyped, or unparseable for one row, that property stays at default and the read continues. For strict reads — fail loudly on any mismatch — use `XlsxReader` directly with `StrictXml = true`.



23 · Cell value extraction

This chapter describes how raw cell values come out of the reader before any type conversion. Useful when you need to understand what `RowReader[index]` returns for a given cell, or when implementing custom mapping logic over the streaming reader.

The OOXML cell type model

OOXML cells carry a `t` attribute that names the cell's type:

OOXML t value	Meaning	RowReader yields
<code>(none)</code> or <code>n</code>	Numeric (default)	<code>double</code> , or <code>DateTime</code> if <code>AutoDetectDates</code> and style is date
<code>s</code>	Shared string (value is the SST index)	<code>string</code> from the SST
<code>str</code>	Inline string (typically formula result)	<code>string</code>
<code>inlineStr</code>	Inline string via <code><is></code> element	<code>string</code>
<code>b</code>	Boolean	<code>bool</code>
<code>e</code>	Error (<code>#NAME?</code> , <code>#VALUE!</code> , etc.)	<code>string</code> with the error code
<code>d</code>	ISO date string	<code>DateTime</code> (or <code>string</code> if unparseable)

Auto-detected dates

When the cell's type is numeric (no `t` attribute) and `AutoDetectDates` is on, the reader checks the cell's style. If the style references a number format code that looks like a date format (`yyyy-mm-dd`, `m/d/yyyy`, `dd-mmm-yy`, etc.), the numeric value is interpreted as an `OADate` and converted to `DateTime`.

This is the same heuristic Excel uses for display. Number format codes are checked against a small list of date-format patterns; anything that includes `y`, `m`, `d`, `h`, `s` in date-like combinations qualifies.

Null cells

An empty cell or a missing cell returns `null`. This is distinct from a cell containing an empty string, which returns `""` (empty string).

Formula cells

For cells with a formula, the reader returns the cached formula result if present (from the `<v>` element inside the cell). If only the formula is present without a cached value, the reader returns `null`.

Files written by Chuvadi.Sheets never include cached formula values — Excel computes them at open time. Files written by Excel typically do include them. The implication: reading a Chuvadi-written file with formulas and then re-reading it after Excel has opened it will give different cell values for the formula cells.

Manual type interpretation

For cases where the auto-detection is not what you want, access the raw value and interpret it yourself:

```
C#
foreach (var row in sheet.Rows)
{
    var raw = row[0];

    if (raw is double serialDate && serialDate > 30000 && serialDate < 70000)
    {
        // Looks like a date serial – interpret as date
        var dt = DateTime.FromOADate(serialDate);
        // ...
    }
    else if (raw is double n)
    {
        // Numeric
    }
    else if (raw is string s)
    {
        // String
    }
}
```



Part V

Protection and encryption



24 · Sheet protection

Sheet protection prevents users from editing locked cells when the file is open in Excel. The password is hashed and stored in the file; users must enter the same password to unprotect.

Basic protection

```
C#
// Streaming writer
sheet.Protect("editpass");

// Workbook model
wb.AddSheet("Data").Protect("editpass");
```

With default options, every cell is locked, and most actions (insert, delete, format, sort) are restricted. The user can still select cells and read their values.

Allowing specific actions

The `SheetProtectionOptions` class exposes fifteen flags, each enabling one user action while protection is active:

```
C#
var options = new SheetProtectionOptions
{
    AllowSelectLockedCells    = true,    // default
    AllowSelectUnlockedCells  = true,    // default
    AllowFormatCells          = false,
    AllowFormatColumns        = false,
    AllowFormatRows           = false,
    AllowInsertColumns        = false,
    AllowInsertRows           = false,
    AllowInsertHyperlinks     = false,
    AllowDeleteColumns        = false,
    AllowDeleteRows           = false,
    AllowSort                  = true,    // allow sorting
    AllowAutoFilter            = true,    // allow filtering
    AllowPivotTables          = false,
    AllowEditObjects           = false,
    AllowEditScenarios         = false,
};

sheet.Protect("editpass", options);
```


The "select" flags are inverted by Excel's XML schema: setting `AllowSelectLockedCells = false` emits `selectLockedCells="1"` in the XML, meaning "selection is restricted." The library handles the inversion; you set the flags as positive permissions.

How the password is stored

The password is hashed using SHA-512 with a random 16-byte salt and a configurable spin count (default 100,000). The salt and hash are stored in the sheet's XML; the password itself is never stored. The verification path repeats the hash with the entered password and compares against the stored hash.

XML

```
<sheetProtection
  algorithmName="SHA-512"
  hashValue="iZNh+4Uh..."
  saltValue="bV3pCp..."
  spinCount="100000"
  sheet="1"
.../>
```

SHEET PROTECTION IS ADVISORY

Sheet protection prevents *well-behaved* users from editing cells through Excel's normal UI. It does not prevent reading — the cell values are stored in plaintext in the xlsx zip. A determined user can strip protection by editing the XML or using any of several free unlock tools. For confidential content, encrypt the workbook (Chapter 26) instead of (or in addition to) protecting sheets.



25 • Workbook structure protection

Workbook structure protection locks the sheet tabs — users cannot add, remove, rename, or reorder sheets in Excel without the password. The window arrangement can optionally be locked too.

Setting it

C#

```
// Workbook model
wb.Protect("structurepass", lockStructure: true, lockWindows: false);

// Streaming writer
writer.ProtectWorkbook("structurepass", lockStructure: true, lockWindows: false);
```

Combining with sheet protection

Workbook structure protection and per-sheet protection are independent and can be used together:

C#

```
var wb = new Workbook();

// Lock the workbook structure
wb.Protect("structurepass", lockStructure: true);

// Also lock individual sheets
var dataSheet = wb.AddSheet("Data");
dataSheet.Protect("editpass-data");

var configSheet = wb.AddSheet("Config");
configSheet.Protect("editpass-config", new SheetProtectionOptions
{
    AllowSort = true,
    AllowAutoFilter = true,
});
```

A user opening this file in Excel needs the structure password to add a sheet, the data sheet's password to edit data cells, and the config sheet's password to edit config cells. Each password is independent — knowing one does not unlock the others.

Storage format

Like sheet protection, the structure password is stored hashed. The XML appears in `workbook.xml`:

XML

```
<workbookProtection
  workbookAlgorithmName="SHA-512"
  workbookHashValue="X4eP..."
  workbookSaltValue="bV3p..."
  workbookSpinCount="100000"
  lockStructure="1"
  lockWindows="0"/>
```



26 · Workbook encryption

Workbook encryption is the only protection mechanism that protects against unauthorised *reading* of the file. The `xlsx` bytes are encrypted with AES-256-CBC; without the password, the content is unrecoverable. This is the appropriate choice for regulated data — patient records, financial details, contractual terms.

Encrypting on save

```
C#
using Chuvadi.Sheets.Excel;

var wb = new Workbook();
var sheet = wb.AddSheet("Confidential");
sheet.Cell("A1").Value = "Patient: Anand Kumar";
sheet.Cell("A2").Value = "Diagnosis: Diabetes Type 2";
sheet.Cell("A3").Value = "Notes: HbA1c 7.8%, on metformin 500mg BD";

wb.SaveTo("confidential.xlsx", new EncryptionOptions
{
    Password = "MyStrongP@ssword!",
    SpinCount = 100_000,
});
```

The output file is a Compound File Binary container holding the encrypted `xlsx` and the metadata Excel needs to verify and decrypt it. Excel prompts for the password on open; entering the correct password produces the plaintext `xlsx` content.

Decrypting on load

```
C#
var wb = Workbook.Load("confidential.xlsx", "MyStrongP@ssword!");

foreach (var sheet in wb.Sheets)
{
    foreach (var cell in sheet.PopulatedCells)
        Console.WriteLine($"{sheet.Name}!{cell.Address}: {cell.Value}");
}
```

If the password is wrong or missing, `XlsxPasswordRequiredException` is thrown with a message indicating which problem occurred.

Choosing the spin count

The spin count controls how many times the password is iterated during key derivation. Higher values make brute-force attacks more expensive but also slow down legitimate opens and saves.

Spin count	Save/open cost	Brute-force cost
10,000	~0.05 s	10× weaker than default
50,000	~0.25 s	2× weaker than default
100,000 (default)	~0.5 s	Matches modern Excel
500,000	~2.5 s	5× stronger
1,000,000	~5 s	10× stronger

For typical business use, the default of 100,000 is appropriate. For high-value files where a slow open is acceptable, increase to 500,000 or 1,000,000. The library refuses spin counts below 50,000 — anything lower is fast enough to brute-force on consumer hardware.

Choosing a password

AES-256 is computationally unbreakable; the only practical attack is to guess the password. Password strength is therefore everything:

- Use at least 12 characters.
- Mix uppercase, lowercase, digits, and symbols.
- Do not reuse passwords from other systems.
- Consider a passphrase — four random words is stronger than most "complex" 8-character passwords.

A password manager is the right tool for generating and storing strong passwords. The library does not enforce password strength — it accepts any non-empty string — but a weak password makes strong encryption pointless.

Excel compatibility verified

The encryption path has been validated against three independent consumers:

- **Microsoft Excel 2016 and later** — opens files end-to-end with the correct password.
- **msoffcrypto-tool** (Python, spec-strict) — decrypts to byte-identical plaintext.
- **LibreOffice Calc 7.x and later** — opens, edits, and re-saves correctly.

Round-tripping through any of these tools preserves the encryption — a file encrypted by Chuvadi.Sheets, opened and re-saved by Excel, remains encrypted with the same algorithm and re-opens via `Workbook.Load`.



27 · Security model

This chapter sets out exactly what each protection mechanism does and does not protect against. Important reading for any application that handles confidential or regulated data.

What each mechanism does

Mechanism	Prevents	Does not prevent
Sheet protection	Editing locked cells in Excel's UI	Reading cell values; stripping via XML edits
Workbook structure protection	Adding/removing/renaming sheets in Excel's UI	Reading any content; modifying via XML edits
Workbook encryption	Reading any content without the password	Brute-force attacks against weak passwords

Threat model

Sheet and structure protection

These are "polite" protections. They prevent legitimate users from inadvertently editing data they should not edit. They do **not** protect against a determined attacker. The file's bytes contain the cell values in plaintext; anyone with file access can read them by:

- Opening the xlsx as a zip and reading `xl/worksheets/sheet1.xml` directly.
- Using a free unlocking tool that removes the `sheetProtection` element from the XML.
- Using `XlsxReader` from this library — it ignores sheet protection by design.

Sheet and structure protection should be used for one purpose: signalling intent to other users of the file, and preventing accidental edits. They are not security.

Workbook encryption

This is real cryptographic protection. AES-256-CBC has no known practical attack; PBKDF2-SHA512 with 100,000 iterations slows brute-force to roughly one attempt every half second on consumer hardware. The HMAC-SHA512 over the encrypted blob prevents tampering — Excel detects bit-flips and refuses to decrypt a modified file.

The remaining weakness is password strength. A 6-character lowercase password can be brute-forced in hours; a 16-character passphrase from a password manager is effectively unbreakable for the foreseeable future. The library can do nothing about password choice; that is the caller's responsibility.

Confidential data checklist

For any application handling regulated or confidential data, the recommended pattern:

1. Use workbook encryption (`EncryptionOptions` on save). Do not rely on sheet protection alone.
2. Generate strong passwords programmatically (e.g. via `RandomNumberGenerator`) — do not let users choose them, since they will choose weak ones.
3. Distribute passwords through a different channel than the file itself.
4. Keep the spin count at default (100,000) or higher.
5. Combine with sheet protection if you also want to signal "do not edit these cells" inside the file.
6. Log access — when the password is requested, who supplied it, when the file was opened.

What the library does not do

- **No certificate-based encryption.** Only password-based. If you need PKI-based key wrapping, this library is not the right tool.
- **No password recovery.** Lose the password, lose the file. The library has no master key, no backdoor, no escrow mechanism.
- **No password complexity enforcement.** Any non-empty string is accepted as a password. The caller must enforce strength policies.
- **No "password to modify" support.** Excel offers a separate "modify" password alongside the "open" password; Chuvadi.Sheets supports only the latter. The "modify" password is widely understood to be advisory anyway.



Part VI

Zip utilities



28 · ZipWriter

The `Chuvadi.Sheets.Zip` namespace exposes a clean API over `System.IO.Compression.ZipArchive`. The same library that produces xlsx files (which are themselves zips internally) also offers a standalone zip API — useful for bundling files alongside the xlsx output, archiving logs, or any general-purpose compression need.

Construction

C#

```
using Chuvadi.Sheets.Zip;
using System.IO.Compression;

// To a file path
using (var zip = ZipWriter.Create("bundle.zip"))
{
    // Add entries
}

// To a stream, leaving it open after dispose
using (var zip = ZipWriter.Create(stream, CompressionLevel.Optimal, leaveOpen:
true))
{
    // ...
}

// With a specific compression level
using (var zip = ZipWriter.Create("bundle.zip", CompressionLevel.Fastest))
{
    // ...
}
```

Compression levels: `NoCompression`, `Fastest`, `Optimal` (default), `SmallestSize`. `Optimal` balances size and time; `SmallestSize` is meaningfully slower for marginal gains.

Adding entries

C#

```
using var zip = ZipWriter.Create("bundle.zip");

// From a file on disk
zip.AddFile("report.xlsx", @"C:\out\report.xlsx");
zip.AddFile("docs/readme.txt", @"C:\out\readme.txt"); // nested path

// From an in-memory stream
using var ms = new MemoryStream(payloadBytes);
zip.AddStream("data.bin", ms);

// From a string
zip.AddText("manifest.json", JsonSerializer.Serialize(manifest));
zip.AddText("readme.txt", "Generated by SIGMA build " + buildId,
    encoding: Encoding.UTF8);

// From raw bytes
zip.AddBytes("snapshot.dat", snapshotBytes);
```

Entry names are normalized to forward slashes regardless of the platform — this matches the zip specification's preference and ensures the archive opens correctly on every platform.

Path-traversal protection on write

The library validates entry names and refuses anything that could be exploited downstream:

- Empty names — `ArgumentException`
- Absolute paths starting with `/` or `\` — `ArgumentException`
- Drive letters (e.g. `C:\file.txt`) — `ArgumentException`
- `..` path segments — `ArgumentException`

This is defense in depth: even if the consumer of the zip does not perform its own validation on extract, an archive produced by `ZipWriter` will not smuggle a path-traversal payload through.

Async writes

C#

```
await using var zip = ZipWriter.Create("bundle.zip");
await zip.AddFileAsync("big-export.xlsx", path, cancellationToken);
await zip.AddStreamAsync("data.bin", contentStream, cancellationToken);
```

Underlying file streams are opened with `FileOptions.Asynchronous` for true non-blocking I/O.



29 · ZipReader

Read-only access to a zip archive, with safe extraction.

Construction

```
C#
using (var zip = ZipReader.Open("bundle.zip"))
{
    // Read entries
}

// From a stream
using (var zip = ZipReader.Open(stream, leaveOpen: true))
{
    // ...
}
```

Enumerating entries

```
C#
foreach (var entry in zip.Entries)
{
    Console.WriteLine($"{entry.Name}");
    Console.WriteLine($"  Size: {entry.Size} bytes ({entry.CompressedSize} com-
pressed)");
    Console.WriteLine($"  Modified: {entry.LastModified}");
    Console.WriteLine($"  Directory: {entry.IsDirectory}");
}
```

Each entry exposes its full name (path inside the archive), uncompressed size, compressed size, last-modified timestamp, and a flag indicating whether it represents a directory (name ends with )

Reading entry content

```
C#
var entry = zip.FindEntry("report.xlsx");
if (entry is not null)
{
    using var s = entry.OpenRead();
    // process the stream
    using var reader = XlsxReader.Open(s);
    foreach (var row in reader.Sheet(1).Rows)
    {
        // ...
    }
}
```

`FindEntry` looks up by exact name (case-sensitive). It returns null if the entry is not found, rather than throwing.

Extracting to disk

```
C#
zip.ExtractTo(@"C:\extracted");
await zip.ExtractToAsync(@"C:\extracted", cancellationToken);
```

This creates the target directory if needed, preserves the archive's directory structure inside, and copies timestamps where possible.

Zip-slip protection on extract

Before writing any entry, the library resolves the entry's full path against the target directory and verifies that the result is inside the target. An entry named `../../../../etc/passwd` would resolve outside the target; the library throws `ZipFormatException` rather than writing.

This protection applies on both Windows (where backslashes complicate path canonicalization) and Unix-like systems. The library normalizes the entry name before checking, so neither `../` nor `..\` nor absolute paths can escape.

IF YOU BUILD YOUR OWN EXTRACTION LOOP

The protection above only applies when you use `ZipReader.ExtractTo`. If you enumerate entries yourself and write them via `entry.OpenRead()` + `File.Open`, you must add your own validation. The pattern is straightforward — combine the target with the entry name, canonicalize, and check that the result is inside the target.



30 · Convenience extensions

The `ZipExtensions` static class adds one-liner methods to `string` and `IEnumerable<string>` for the most common operations.

Zip a directory tree

```
C#
"C:\source\folder".ZipDirectory(@"C:\out\bundle.zip");

// With a specific compression level
"C:\source\folder".ZipDirectory(@"C:\out\bundle.zip", CompressionLevel.Fastest);

// Async
await "C:\source\folder".ZipDirectoryAsync(@"C:\out\bundle.zip",
    CompressionLevel.Optimal, cancellationToken);
```

Entries are named relative to the source directory, using forward slashes. Subdirectories are preserved.

Zip a list of files

```
C#
string[] files = { @"C:\out\report.xlsx", @"C:\out\summary.pdf", @"C:\out\log.txt" };
files.ToZip(@"C:\out\bundle.zip");
```

Each file appears in the archive under its basename (e.g. `report.xlsx`, not `C:\out\report.xlsx`). For full control over entry names, use `ZipWriter` directly.

Extract

```
C#
"C:\in\bundle.zip".ExtractTo(@"C:\out\extracted");

// Async
await "C:\in\bundle.zip".ExtractToAsync(@"C:\out\extracted", cancellationToken);
```


List entries without extracting

C#

```
foreach (var entry in @"C:\in\bundle.zip".ListZipEntries())  
{  
    Console.WriteLine($"{entry.Name} — {entry.Size} bytes");  
}
```

This opens the archive, snapshots the entry list, and disposes the archive before returning. Use when you only need metadata, not the entry contents.



Part VII

Performance and operations



31 · Performance characteristics

Indicative numbers for the v1.0 library on a typical developer laptop (.NET 10, Intel i7, NVMe SSD, default options). These are illustrative — exact figures depend on hardware, OS, and the shape of the data.

Write throughput

Dataset	Configuration	Time	Output size
10,000 rows × 6 columns	Streaming writer, shared strings	~0.2 s	~250 KB
100,000 rows × 6 columns	Streaming writer, shared strings	~1.8 s	~2.1 MB
1,000,000 rows × 6 columns	Streaming writer, shared strings	~18 s	~21 MB
1,000,000 rows × 6 columns	Streaming writer, inline strings	~14 s	~38 MB
1,000,000 rows × 6 columns	Workbook model	~28 s	~21 MB

Read throughput

Dataset	Method	Time
1,000,000 rows × 6 columns	Streaming reader (RowReader)	~12 s
1,000,000 rows × 6 columns	<code>.ReadXlsx<T>()</code> into typed list	~18 s
1,000,000 rows × 6 columns	<code>.ReadXlsx()</code> into DataSet	~22 s

Memory profile

Operation	Peak working set
Streaming write, 1M rows	~40 MB (mostly the shared-string table)
Streaming write, 1M rows, inline strings	~8 MB
Streaming read, 1M rows	~25 MB (SST + styles + row buffer)
Workbook.Load on 1M-row file	~280 MB (every cell materialised)

Encryption overhead

Encrypting a file adds the cost of one full pass through AES-256-CBC plus the PBKDF2 key derivation (which is the bulk of the cost at the default 100,000 spin count):

File size	Save time (plain)	Save time (encrypted)
250 KB	~0.2 s	~0.7 s
2 MB	~1.8 s	~2.3 s
21 MB	~18 s	~19 s

The encryption overhead is roughly constant (the PBKDF2 step) plus a few percent for the AES pass. For large files, encryption is essentially free; for small files, it adds half a second.



32 · Streaming vs random-access

The choice between `XlsxWriter` (streaming) and `Workbook` (random-access) is the most significant performance decision when using this library. The trade-off:

Concern	Streaming	Random-access
Memory	Bounded — flat regardless of row count	Proportional to populated cells
Write speed	Faster (no model overhead)	Slower (model is built then serialized)
Row order	Must be sequential	Any order
Cell addressing	Implicit (column index advances)	Explicit (A1 or row/column)
Mutation after write	Not possible	Trivial
Mixing fluent and direct cells	Awkward	Natural
Code clarity for simple cases	More verbose	More concise

When streaming wins

- Dataset of unknown or potentially large size — database query results, log exports.
- Memory-constrained environments — containers with limits, serverless functions.
- Long-running services where peak memory matters more than per-call latency.
- Sequential writes that follow the row order naturally.

When random-access wins

- Files with fewer than ~10,000 cells where memory is not a concern.
- Layouts where you build the structure first and fill in values later.
- Sparse data — many cells empty, only some populated.
- Workflows that load, mutate, and re-save existing files.
- Calculations that need to back-reference earlier cells while writing.

The hybrid case

For some use cases — small summary sheets alongside large data sheets — combining the two through `XlsxExport` works well:

C#

```
// Build small sheets in-memory; stream the big one
new XlsxExport("report.xlsx")
    .AddSheet("Summary", summaryRows)           // small, ergonomic export
    .AddSheet("Detail", detailRowsEnumerable)    // streaming over 1M rows
    .Save();
```

Each `AddSheet` call streams into the writer; the source can be anything that implements `IEnumerable<T>`, including a generator method that yields one row at a time.



33 • Thread safety and concurrency

Chuvadi.Sheets's threading model is conservative: **no public type is thread-safe**. Use one instance per thread, or guard with your own synchronisation.

What this means in practice

Operation	Threading guarantee
Two threads writing to the same <code>XlsxWriter</code>	Undefined behaviour; do not do this
Two threads writing to different <code>SheetWriter</code> s of the same <code>XlsxWriter</code>	Undefined behaviour; sheet writers share the parent's state
Two threads each owning their own <code>XlsxWriter</code>	Safe — independent state
Two threads reading the same <code>XlsxReader</code>	Undefined behaviour
Two threads enumerating different sheets of the same reader	Undefined behaviour — reader state is shared
Two threads each owning their own <code>XlsxReader</code> over the same file	Safe — they will open separate file streams
One thread building a <code>CellStyle</code> while another writes	Safe — <code>CellStyle</code> is an immutable value type

Async is single-threaded

The async APIs (`WriteRowAsync`, `SaveAsync`, `ExtractToAsync`) make I/O non-blocking but do not introduce concurrency. A single async method may suspend and resume on different threads, but only one continuation runs at a time. The same "one instance per thread" rule applies — do not run two async calls against the same writer concurrently.

Producer/consumer patterns

For high-throughput pipelines, the recommended pattern is one thread/task per writer, fed by a queue:

```
C#
var queue = new BlockingCollection<Patient>(boundedCapacity: 1000);

// Producer task – populates the queue
_ = Task.Run(() =>
{
    foreach (var p in QueryDatabaseStream())
        queue.Add(p);
    queue.CompleteAdding();
});

// Consumer – writes to xlsx
using var writer = XlsxWriter.Create("export.xlsx");
var sheet = writer.AddSheet("Patients");
sheet.WriteRow("Id", "Name", "Dob", "Active");

foreach (var p in queue.GetConsumingEnumerable())
{
    sheet.WriteRow(p.Id, p.Name, p.Dob, p.Active);
}
writer.Save();
```

This decouples I/O from the source — the database query can produce at its own pace while the writer flushes at its own pace.



Part VIII

Reference



34 · Public API summary

A condensed catalogue of every public type in `Chuvadi.Sheets.Excel` and `Chuvadi.Sheets.Zip`. For full descriptions see the body chapters of this guide; this is a lookup reference.

Workbook model (Chapter 11)

Member	Description
<code>new Workbook()</code>	Create an empty workbook
<code>Workbook.AddSheet(name)</code>	Append a new sheet, return it
<code>Workbook.Sheets</code>	<code>ReadOnlyList<Sheet></code>
<code>Workbook[name]</code>	Get sheet by name (case-insensitive)
<code>Workbook.DefinedNames</code>	<code>DefinedNameCollection</code>
<code>Workbook.Protect(pwd, lockStructure, lockWindows)</code>	Workbook structure protection
<code>Workbook.SaveTo(path)</code>	Save to file
<code>Workbook.SaveTo(stream)</code>	Save to stream
<code>Workbook.SaveTo(path, encryptionOptions)</code>	Save encrypted
<code>Workbook.Load(path)</code> (static)	Load from file
<code>Workbook.Load(path, password)</code> (static)	Load (possibly encrypted)
<code>Workbook.Load(stream)</code> (static)	Load from stream

Sheet member	Description
<code>Sheet.Name</code>	The sheet's name
<code>Sheet.Cell(a1Address)</code>	Get or create a cell
<code>Sheet.Cell(row, column)</code>	Get or create by (row, col)
<code>Sheet.Range(a1Range)</code>	Range descriptor (supports <code>.Merge()</code>)
<code>Sheet.Columns[n]</code>	Per-column accessor (<code>.Width</code>)
<code>Sheet.PopulatedCells</code>	Enumerate non-empty cells
<code>Sheet.FreezeRows(n)</code>	Freeze the first n rows
<code>Sheet.FreezeColumns(n)</code>	Freeze the first n columns
<code>Sheet.MergeCells(range)</code>	Add a merge range
<code>Sheet.AutoFilter(range)</code>	Set autofilter
<code>Sheet.AddDataValidation(range, rule)</code>	Add validation
<code>Sheet.AddConditionalFormat(range, rule)</code>	Add conditional format
<code>Sheet.Protect(password, options?)</code>	Protect with optional flags
Cell member	Description
<code>Cell.Row, Cell.Column, Cell.Address</code>	Position
<code>Cell.Value</code> (get/set)	Cell value (clears Formula when set)
<code>Cell.Formula</code> (get/set)	Formula text without leading '=' (clears Value)
<code>Cell.Style</code> (get/set)	Optional cell style

Streaming writer (Chapter 12)

Member	Description
<code>XlsxWriter.Create(path, options?)</code> (static)	Open for write
<code>XlsxWriter.Create(stream, options?)</code> (static)	Open for write to stream
<code>XlsxWriter.AddSheet(name)</code>	Append sheet, return Sheet-Writer
<code>XlsxWriter.SheetCount</code>	Number of sheets added
<code>XlsxWriter.SetDefinedNames(names)</code>	Workbook-scoped defined names
<code>XlsxWriter.ProtectWorkbook(pwd, lockStructure, lockWindows)</code>	Workbook protection
<code>XlsxWriter.Save()</code>	Finalize and write
<code>XlsxWriter.SaveAsync(ct?)</code>	Async finalize
<code>XlsxWriter.Dispose()</code>	Clean up temp files

SheetWriter member	Description
<code>SheetWriter.Name</code>	Sheet name
<code>SheetWriter.RowsWritten</code>	Count of rows written
<code>SheetWriter.SetColumnWidth(col, width)</code>	Single column (before first row)
<code>SheetWriter.SetColumnWidth(from, to, width)</code>	Range of columns (before first row)
<code>SheetWriter.FreezeRows(n)</code>	Freeze rows (before first row)
<code>SheetWriter.FreezeColumns(n)</code>	Freeze columns (before first row)
<code>SheetWriter.WriteHeader(strings)</code>	Write a header row
<code>SheetWriter.WriteRow(values)</code>	Write a row of typed values
<code>SheetWriter.WriteRow(builderAction)</code>	Fluent per-cell write
<code>SheetWriter.WriteHeaderAsync / WriteRowAsync</code>	Async equivalents
<code>SheetWriter.MergeCells(range)</code>	After-data: merge range
<code>SheetWriter.AutoFilter(range)</code>	After-data: autofilter
<code>SheetWriter.AddDataValidation(range, rule)</code>	After-data: validation
<code>SheetWriter.AddConditionalFormat(range, rule)</code>	After-data: conditional format
<code>SheetWriter.AddTable(range, name, style, headers)</code>	After-data: structured table
<code>SheetWriter.Protect(pwd, options?)</code>	After-data: sheet protection

Streaming reader (Chapters 20-23)

Member	Description
<code>XlsxReader.Open(path, options?)</code> (static)	Open file for read
<code>XlsxReader.Open(stream, options?)</code> (static)	Open stream for read
<code>XlsxReader.Sheets</code>	<code>IReadOnlyList<SheetInfo></code>
<code>XlsxReader.Sheet(name)</code>	Get SheetReader by name
<code>XlsxReader.Sheet(index1Based)</code>	Get SheetReader by 1-based index
<code>SheetReader.Name</code>	Sheet name
<code>SheetReader.Rows</code>	<code>IEnumerable<RowReader></code>
<code>SheetReader.HeaderIndex</code>	<code>Dictionary<string, int>?</code>
<code>RowReader.Count</code>	Number of cells in this row
<code>RowReader.RowNumber</code>	1-based row number in the sheet
<code>RowReader[col0]</code>	Raw value by 0-based column
<code>RowReader[name]</code>	Raw value by header name
<code>RowReader.GetString/GetInt32/GetInt64/GetDouble/GetDecimal/GetBool/GetDateTime</code>	Typed getters (throw on missing/wrong type)
<code>RowReader.Get<T>(col or name)</code>	Generic typed getter
<code>RowReader.TryGet<T>(col or name, out value)</code>	Non-throwing getter

Cell styling (Chapter 14)

Type/member	Description
<code>CellStyle</code> (readonly record struct)	Immutable style; value equality for dedup
<code>CellStyle.Default</code>	The default (no styling)
<code>CellStyle.IsDefault</code>	True if all fields at default
<code>new CellStyleBuilder()</code>	Start a fluent style build
<code>.Font(name, size)</code>	Set font name and size
<code>.FontName(name) / .FontSize(size)</code>	Set individually
<code>.Bold(true) / .Italic(true) / .Underline(true)</code>	Type style flags
<code>.Foreground(hex)</code>	Font color
<code>.Background(hex)</code>	Fill color
<code>.Format(code)</code>	Number format code
<code>.BorderAll(style, color)</code>	All four edges
<code>.BorderTop/Bottom/Left/Right(style, color)</code>	Per edge
<code>.HAlign(value) / .VAlign(value)</code>	Horizontal / vertical alignment
<code>.WrapText(true)</code>	Wrap text in cell
<code>.Build()</code>	Materialize the immutable <code>CellStyle</code>

Ergonomic exports (Chapter 15)

Member	Description
<code>list.ToXlsx(path)</code>	Single-sheet export, reflection on T
<code>list.ToXlsx(path, configure)</code>	With fluent config
<code>list.ToXlsx(stream, configure?)</code>	To stream
<code>table.ToXlsx(path, configure?)</code>	DataTable export
<code>new XlsxExport(path).AddSheet(...).Save()</code>	Multi-sheet export
<code>XlsxExportConfig<T>.SheetName / HeaderStyle / AutoFilter / FreezeHeaderRow / AsTable</code>	Common settings
<code>.Column(expression, header?, width?, format?, style?)</code>	Per-column override
<code>.Ignore(expression)</code>	Exclude property
<code>.ColumnStyle(expression, style)</code>	Apply style to column

Ergonomic reads (Chapter 22)

Member	Description
<code>path.ReadXlsx()</code>	Read all sheets to DataSet
<code>path.ReadXlsxSheet(name?)</code>	Read a sheet to DataTable
<code>path.ReadXlsx<T>(name?)</code>	Read to List<T>

Attributes (Chapter 15)

Attribute	Use
<code>[Column("header", Order=, Width=, Format=)]</code>	Customize property column
<code>[ColumnIgnore]</code>	Exclude property from export
<code>[ColumnStyle(Bold=, Italic=, Background=, Foreground=, Format=)]</code>	Per-cell style for column

Data validation (Chapter 17)

Factory	Description
<code>DataValidation.List(values)</code>	Dropdown of literal values (≤ 255 chars)
<code>DataValidation.ListFromRange(range)</code>	Dropdown from sheet range or name
<code>DataValidation.WholeNumber(min?, max?)</code>	Integer range
<code>DataValidation.DecimalNumber(min?, max?)</code>	Decimal range
<code>DataValidation.Date(min?, max?)</code>	Date range
<code>DataValidation.TextLength(min?, max?)</code>	String length

Conditional formatting (Chapter 18)

Factory	Description
<code>ConditionalFormat.ColorScale.RedWhiteGreen / GreenWhiteRed / Blue-WhiteRed</code>	Three-color scales
<code>ConditionalFormat.ColorScale.Custom(min, mid, max)</code>	Custom scale
<code>ConditionalFormat.DataBar.Blue / Green / Red</code>	Built-in data bars
<code>ConditionalFormat.DataBar.Custom(color)</code>	Custom data bar
<code>ConditionalFormat.CellIs(op, threshold, style)</code>	Comparison rule
<code>ConditionalFormat.Between(min, max, style)</code>	Range rule

Zip API (Chapters 28-30)

Member	Description
<code>ZipWriter.Create(path, level?)</code>	Open zip for write
<code>ZipWriter.Create(stream, level?, leaveOpen?)</code>	Open zip on stream
<code>ZipWriter.AddFile(name, path)</code>	Add file from disk
<code>ZipWriter.AddStream(name, stream)</code>	Add from stream
<code>ZipWriter.AddText(name, text, encoding?)</code>	Add UTF-8 text
<code>ZipWriter.AddBytes(name, bytes)</code>	Add raw bytes
<code>ZipWriter.AddFileAsync</code> / <code>AddStreamAsync</code>	Async variants
<code>ZipReader.Open(path)</code>	Open for read
<code>ZipReader.Entries</code>	List entries
<code>ZipReader.FindEntry(name)</code>	Look up by exact name
<code>ZipReader.ExtractTo(dir)</code>	Safe extract (zip-slip protected)
<code>ZipReader.ExtractToAsync(dir, ct?)</code>	Async extract
<code>dir.ZipDirectory(zipPath, level?)</code>	Zip a directory tree
<code>files.ToZip(zipPath, level?)</code>	Zip a list of files
<code>zipPath.ExtractTo(dir)</code>	Extract a zip file
<code>zipPath.ListZipEntries()</code>	List entries by path



35 • Exception types

The library defines two exception types, both sealed and serializable.

XlsxFormatException

Thrown when an xlsx file is structurally invalid — missing required parts, malformed XML in critical content, or content that cannot be interpreted. The reader is forgiving about minor deviations (unknown attributes, missing optional elements, foreign-file conventions); it throws when continuing would cause silent data loss.

```
C#
try
{
    using var reader = XlsxReader.Open("input.xlsx");
    // ...
}
catch (XlsxFormatException ex)
{
    Console.Error.WriteLine($"Bad xlsx: {ex.Message}");
    // Maybe report to user; do not retry
}
```

XlsxPasswordRequiredException

Thrown when attempting to read an encrypted xlsx without a password, or with the wrong password.

```
C#
try
{
    var wb = Workbook.Load("secret.xlsx", enteredPassword);
}
catch (XlsxPasswordRequiredException ex)
{
    // Prompt user for password and retry
}
```

ZipFormatException

Thrown when a zip archive is malformed or violates safety constraints (e.g. a zip-slip entry on extract).

BCL exceptions you may also see

- `ArgumentException` — invalid arguments (bad color, bad name, malformed range, etc.)
- `ArgumentNullException` — null where not allowed
- `ArgumentOutOfRangeException` — value outside expected range (e.g. row index < 1)
- `InvalidOperationException` — wrong state (writing a row before setting column widths, calling Save twice)
- `FileNotFoundException` / `DirectoryNotFoundException` — paths that don't exist
- `UnauthorizedAccessException` — file permissions
- `IOException` — disk full, network drive disconnected
- `KeyNotFoundException` — sheet name not in workbook
- `ObjectDisposedException` — using a disposed writer or reader
- `InvalidCastException` — typed getter on wrong cell type



36 · Options and configuration

XlsxWriterOptions

UseInlineStrings (bool, default false)

If true, write strings inline rather than through the shared-strings table. Lower memory, larger files.

TempDirectory (string?, default Path.GetTempPath())

Where per-sheet temp files are created during streaming write.

DefaultDateFormat (string, default "yyyy-mm-dd")

Number format used for date-only `DateTime` values when no style is supplied.

DefaultDateTimeFormat (string, default "yyyy-mm-dd hh:mm:ss")

Number format used for `DateTime` values with non-zero time component when no style is supplied.

DefaultTimeSpanFormat (string, default "[h]:mm:ss")

Number format used for `TimeSpan` values when no style is supplied. The bracketed hours prevent truncation of durations greater than 24 hours.

XlsxReaderOptions

TreatFirstRowAsHeaders (bool, default true)

If true, the first row is consumed and used to build the name → column-index map. Disable when reading sheets without headers.

AutoDetectDates (bool, default true)

If true, numeric cells with a date-like number format return as `DateTime` rather than `double`.

StrictXml (bool, default false)

If true, malformed XML in any reachable sheet throws `XlsxFormatException`. If false, broken sheets are silently dropped.

EncryptionOptions

Password (string, required)

Password required to open the file. Must not be empty.

SpinCount (int, default 100,000)

PBKDF2 iteration count for key derivation. Higher = slower brute force but slower open. Minimum 50,000.

SheetProtectionOptions

Fifteen flags controlling what users may do while a sheet is protected. All are settable independently; defaults match Excel's "Protect Sheet" dialog defaults:

```
AllowSelectLockedCells    = true
AllowSelectUnlockedCells  = true
AllowFormatCells          = false
AllowFormatColumns        = false
AllowFormatRows           = false
AllowInsertColumns        = false
AllowInsertRows           = false
AllowInsertHyperlinks     = false
AllowDeleteColumns        = false
AllowDeleteRows           = false
AllowSort                  = false
AllowAutoFilter            = false
AllowPivotTables           = false
AllowEditObjects           = false
AllowEditScenarios        = false
```



Part IX

Appendices



Appendix A • Complete example — patient register

A complete program that builds a patient register `xlsx` with styled header, frozen header row, autofilter, column widths, data validation for a status column, and conditional formatting on age. Compiles and runs as-is in a .NET 10 console project with a reference to `Chuvadi.Sheets`.

C#

```
using System;
using System.Collections.Generic;
using Chuvadi.Sheets.Excel;

namespace PatientRegisterExample;

public enum PatientStatus { Active, Inactive, Discharged, Deceased }

public class Patient
{
    public string Id { get; set; } = "";
    public string Name { get; set; } = "";
    public DateTime DateOfBirth { get; set; }
    public string Gender { get; set; } = "";
    public string Phone { get; set; } = "";
    public string Address { get; set; } = "";
    public PatientStatus Status { get; set; }
    public DateTime LastVisit { get; set; }

    public int AgeYears(DateTime asOf) => (int)((asOf - DateOfBirth).TotalDays /
365.25);
}

internal static class Program
{
    private static IEnumerable<Patient> SamplePatients() => new[]
    {
        new Patient { Id = "P001", Name = "Anand Kumar",      DateOfBirth =
new(1962,  4, 12),
                    Gender = "M", Phone = "+91 98 4567 1230", Address = "MG Road,
Bangalore",
                    Status = PatientStatus.Active,      LastVisit = new(2026, 4,
18) },
        new Patient { Id = "P002", Name = "Priya Sharma",      DateOfBirth =
new(1979, 11,  3),
                    Gender = "F", Phone = "+91 98 2233 1144", Address = "Andheri
West, Mumbai",
                    Status = PatientStatus.Active,      LastVisit = new(2026, 5,
12) },
        new Patient { Id = "P003", Name = "Ravi Iyer",          DateOfBirth =
new(1985,  2, 27),
                    Gender = "M", Phone = "+91 99 7788 6655", Address = "Adyar,
Chennai",
                    Status = PatientStatus.Inactive,    LastVisit = new(2024, 9,
30) },
        new Patient { Id = "P004", Name = "Saira Khan",        DateOfBirth =
new(1990,  6, 15),
                    Gender = "F", Phone = "+91 96 1122 3344", Address = "Connaught
Place, Delhi",
                    Status = PatientStatus.Active,      LastVisit = new(2026, 5,
21) },
        new Patient { Id = "P005", Name = "Vikram Singh",      DateOfBirth =
new(1955,  8,  9),
```

```

        Gender = "M", Phone = "+91 95 5566 7788", Address = "Sector
17, Chandigarh",
        Status = PatientStatus.Discharged, LastVisit = new(2025, 12,
4) },
        new Patient { Id = "P006", Name = "Meera Nair",        DateOfBirth =
new(1972, 1, 22),
        Gender = "F", Phone = "+91 94 3344 5566", Address = "Kochi,
Kerala",
        Status = PatientStatus.Active,        LastVisit = new(2026, 5,
8) },
    };

private static void Main()
{
    const string outputPath = "patients.xlsx";
    var patients = new List<Patient>(SamplePatients());
    var today = DateTime.Today;

    // ---- Styles -----
    var headerStyle = new CellStyleBuilder()
        .Font("Calibri", 11).Bold()
        .Background("#1A237E").Foreground("#FFFFFF")
        .HAlign(HorizontalAlign.Center).VAlign(VerticalAlign.Center)
        .BorderBottom(BorderStyle.Medium, "#B8860B")
        .Build();

    var titleStyle = new CellStyleBuilder()
        .Font("Calibri", 16).Bold()
        .Foreground("#1A237E")
        .HAlign(HorizontalAlign.Center)
        .Build();

    var ageWarnStyle = new CellStyleBuilder()
        .Bold().Foreground("#C62828")
        .Build();

    // ---- Write -----
    using var writer = XlsxWriter.Create(outputPath);
    var sheet = writer.AddSheet("Patients");

    // Column widths must be set before the first row
    sheet
        .SetColumnWidth(1, 10)    // Id
        .SetColumnWidth(2, 24)    // Name
        .SetColumnWidth(3, 14)    // DateOfBirth
        .SetColumnWidth(4, 6)     // Age
        .SetColumnWidth(5, 8)     // Gender
        .SetColumnWidth(6, 20)    // Phone
        .SetColumnWidth(7, 32)    // Address
        .SetColumnWidth(8, 12)    // Status
        .SetColumnWidth(9, 14)    // LastVisit
        .FreezeRows(2);           // Title + header

    // Title row spanning all columns

```

```

sheet.WriteRow(row => row.Cell("Patient Register – May 2026", titleStyle));
sheet.MergeCells("A1:I1");

// Header row
sheet.WriteRow(row => row
    .Cell("ID", headerStyle)
    .Cell("Name", headerStyle)
    .Cell("DoB", headerStyle)
    .Cell("Age", headerStyle)
    .Cell("Gender", headerStyle)
    .Cell("Phone", headerStyle)
    .Cell("Address", headerStyle)
    .Cell("Status", headerStyle)
    .Cell("Last Visit", headerStyle));

// Data rows
foreach (var p in patients)
{
    var age = p.AgeYears(today);
    sheet.WriteRow(row => row
        .Cell(p.Id)
        .Cell(p.Name)
        .Cell(p.DateOfBirth)
        .Cell(age, age >= 65 ? ageWarnStyle : CellStyle.Default)
        .Cell(p.Gender)
        .Cell(p.Phone)
        .Cell(p.Address)
        .Cell(p.Status.ToString())
        .Cell(p.LastVisit));
}

// After-data features
sheet.AutoFilter($"A2:I{sheet.RowsWritten}");

// Status column: dropdown with allowed values
sheet.AddDataValidation(
    $"H3:H{sheet.RowsWritten + 100}",
    DataValidation.List("Active", "Inactive", "Discharged", "Deceased"));

// Age column: red gradient on high values
sheet.AddConditionalFormat(
    $"D3:D{sheet.RowsWritten}",
    ConditionalFormat.ColorScale.GreenWhiteRed);

// Wrap the data as an Excel table
sheet.AddTable(
    $"A2:I{sheet.RowsWritten}",
    "PatientRegister",
    TableStyle.Medium2,
    "ID", "Name", "DoB", "Age", "Gender", "Phone", "Address", "Status",
    "Last Visit");

writer.Save();

```

```

        Console.WriteLine($"Wrote {outputPath} – {patients.Count} patients.");

        // ---- Read back -----
        using var reader = XlsxReader.Open(outputPath, new XlsxReaderOptions
        {
            TreatFirstRowAsHeaders = false, // We have a title row + header row
        });

        var dataSheet = reader.Sheet("Patients");
        int rowCount = 0;
        foreach (var row in dataSheet.Rows)
        {
            rowCount++;
            if (rowCount <= 2) continue; // skip title and header
            Console.WriteLine(
                $"Row {row.RowNumber}: {row[0]} {row[1],-20} age {row[3]}
                ({row[7]})");
        }
    }
}

```

Expected output:

Shell

```

Wrote patients.xlsx – 6 patients.
Row 3: P001  Anand Kumar           age 64  (Active)
Row 4: P002  Priya Sharma          age 46  (Active)
Row 5: P003  Ravi Iyer             age 41  (Inactive)
Row 6: P004  Saira Khan            age 35  (Active)
Row 7: P005  Vikram Singh          age 70  (Discharged)
Row 8: P006  Meera Nair            age 54  (Active)

```



Appendix B • Complete example — financial report

A monthly financial report with section headers, formula cells for subtotals and totals, currency formatting, and conditional formatting on margin percentages.

C#

```

using System;
using System.Globalization;
using Chuvadi.Sheets.Excel;

namespace FinancialReportExample;

internal static class Program
{
    private static void Main()
    {
        const string outputPath = "financial-report.xlsx";

        // ---- Styles -----
        var titleStyle = new CellStyleBuilder()
            .Font("Calibri", 18).Bold()
            .Foreground("#1A237E")
            .Build();

        var sectionStyle = new CellStyleBuilder()
            .Font("Calibri", 12).Bold()
            .Background("#B8860B").Foreground("#FFFFFF")
            .HAlign(HorizontalAlign.Left)
            .Build();

        var headerStyle = new CellStyleBuilder()
            .Bold().Background("#1A237E").Foreground("#FFFFFF")
            .HAlign(HorizontalAlign.Center)
            .BorderBottom(BorderStyle.Medium, "#B8860B")
            .Build();

        var moneyStyle = new CellStyleBuilder()
            .Format("\"₹\"#,##0.00")
            .HAlign(HorizontalAlign.Right)
            .Build();

        var totalStyle = new CellStyleBuilder()
            .Bold().Background("#F0E9D0")
            .Format("\"₹\"#,##0.00")
            .BorderTop(BorderStyle.Medium, "#1A237E")
            .BorderBottom(BorderStyle.Double, "#1A237E")
            .HAlign(HorizontalAlign.Right)
            .Build();

        var percentStyle = new CellStyleBuilder()
            .Format("0.0%")
            .HAlign(HorizontalAlign.Right)
            .Build();

        // ---- Write -----
        using var writer = XlsxWriter.Create(outputPath);
        var sheet = writer.AddSheet("Q2 2026");

        sheet

```

```

        .SetColumnWidth(1, 32)           // Item description
        .SetColumnWidth(2, 16)          // Apr
        .SetColumnWidth(3, 16)          // May
        .SetColumnWidth(4, 16)          // Jun
        .SetColumnWidth(5, 16)          // Quarter total
        .SetColumnWidth(6, 12)          // % of revenue
        .FreezeRows(3);

// Title
sheet.WriteRow(row => row.Cell("Quarterly Financial Report", titleStyle));
sheet.MergeCells("A1:F1");
sheet.WriteRow(row => row.Cell("Quarter 2 – April to June 2026", Cell-
Style.Default));
sheet.MergeCells("A2:F2");

// Header row
sheet.WriteRow(row => row
    .Cell("Line item", headerStyle)
    .Cell("April",      headerStyle)
    .Cell("May",        headerStyle)
    .Cell("June",       headerStyle)
    .Cell("Q2 total",   headerStyle)
    .Cell("% revenue",  headerStyle));

// ---- Revenue section -----
sheet.WriteRow(row => row.Cell("REVENUE", sectionStyle));
sheet.MergeCells($"A{sheet.RowsWritten}:F{sheet.RowsWritten}");

    WriteLineItem(sheet, "Consulting services", 425000m, 467000m, 512000m,
moneyStyle, percentStyle);
    WriteLineItem(sheet, "Software licenses",   180000m, 195000m, 210000m,
moneyStyle, percentStyle);
    WriteLineItem(sheet, "Support contracts",    78000m,  78000m,  85000m,
moneyStyle, percentStyle);

var revenueStart = sheet.RowsWritten - 2;
var revenueEnd = sheet.RowsWritten;

sheet.WriteRow(row => row
    .Cell("Revenue total", new CellStyleBuilder().Bold().Build())
    .Formula($"SUM(B{revenueStart}:B{revenueEnd})", totalStyle)
    .Formula($"SUM(C{revenueStart}:C{revenueEnd})", totalStyle)
    .Formula($"SUM(D{revenueStart}:D{revenueEnd})", totalStyle)
    .Formula($"SUM(B{sheet.RowsWritten + 1}:D{sheet.RowsWritten + 1})",
totalStyle)
    .Cell(1.0, percentStyle));
var revenueTotalRow = sheet.RowsWritten;

// Spacer
sheet.WriteRow(row => row.Cell(""));

// ---- Costs section -----
sheet.WriteRow(row => row.Cell("COST OF SALES", sectionStyle));
sheet.MergeCells($"A{sheet.RowsWritten}:F{sheet.RowsWritten}");

```

```

        WriteLineItemWithPercentFormula(sheet, "Cloud infrastructure", 42000m,
48000m, 52000m, revenueTotalRow, moneyStyle, percentStyle);
        WriteLineItemWithPercentFormula(sheet, "Subcontractors",          95000m,
105000m, 122000m, revenueTotalRow, moneyStyle, percentStyle);
        WriteLineItemWithPercentFormula(sheet, "Software licenses",      18000m,
18000m, 22000m, revenueTotalRow, moneyStyle, percentStyle);
        WriteLineItemWithPercentFormula(sheet, "Marketing",              8000m,
15000m, 20000m, revenueTotalRow, moneyStyle, percentStyle);

        var costsStart = sheet.RowsWritten - 3;
        var costsEnd = sheet.RowsWritten;

        sheet.WriteRow(row => row
            .Cell("Costs total", new CellStyleBuilder().Bold().Build())
            .Formula($"SUM(B{costsStart}:B{costsEnd})", totalStyle)
            .Formula($"SUM(C{costsStart}:C{costsEnd})", totalStyle)
            .Formula($"SUM(D{costsStart}:D{costsEnd})", totalStyle)
            .Formula($"SUM(B{sheet.RowsWritten + 1}:D{sheet.RowsWritten + 1})",
totalStyle)
            .Formula($"E{sheet.RowsWritten + 1}/E{revenueTotalRow}", percentStyle));
        var costsTotalRow = sheet.RowsWritten;

        // Spacer
        sheet.WriteRow(row => row.Cell(""));

        // ---- Gross margin -----
        sheet.WriteRow(row => row
            .Cell("GROSS MARGIN", sectionStyle)
            .Formula($"B{revenueTotalRow}-B{costsTotalRow}", totalStyle)
            .Formula($"C{revenueTotalRow}-C{costsTotalRow}", totalStyle)
            .Formula($"D{revenueTotalRow}-D{costsTotalRow}", totalStyle)
            .Formula($"E{revenueTotalRow}-E{costsTotalRow}", totalStyle)
            .Formula($"E{sheet.RowsWritten + 1}/E{revenueTotalRow}", percentStyle));

        // Margin column: color-scale to highlight strong/weak months
        sheet.AddConditionalFormat($"F{sheet.RowsWritten}",
            ConditionalFormat.CellIs(ComparisonOp.LessThan, 0.30,
                new CellStyleBuilder().Bold().Foreground("#C62828").Build()));

        writer.Save();
        Console.WriteLine($"Wrote {outputPath}");
    }

    private static void WriteLineItem(SheetWriter sheet, string label,
        decimal apr, decimal may, decimal jun, CellStyle money, CellStyle pct)
    {
        sheet.WriteRow(row => row
            .Cell(label)
            .Cell(apr, money)
            .Cell(may, money)
            .Cell(jun, money)
            .Formula($"SUM(B{sheet.RowsWritten + 1}:D{sheet.RowsWritten + 1})",
money)

```



```
        .Cell("")); // % column filled by the formula version below
    }

    private static void WriteLineItemWithPercentFormula(SheetWriter sheet, string
label,
        decimal apr, decimal may, decimal jun, int revenueTotalRow,
        CellStyle money, CellStyle pct)
    {
        var thisRow = sheet.RowsWritten + 1;
        sheet.WriteRow(row => row
            .Cell(label)
            .Cell(apr, money)
            .Cell(may, money)
            .Cell(jun, money)
            .Formula($"SUM(B{thisRow}:D{thisRow})", money)
            .Formula($"E{thisRow}/E{revenueTotalRow}", pct));
    }
}
```

The report opens in Excel with all formulas computed, currency in rupee format, percentages rounded to one decimal, and the gross margin row highlighted in red if it drops below 30%.



Appendix C • Complete example — encrypted backup

A program that encrypts a workbook for archival storage and a separate program that decrypts and verifies it. The pattern is useful for regulatory backups, audit trails, and any scenario where a file must be transportable but unreadable without a key.

encrypt-backup.cs

C#

```
using System;
using System.IO;
using System.Security.Cryptography;
using System.Text;
using Chuvadi.Sheets.Excel;

namespace EncryptedBackupExample;

internal static class Encrypt
{
    private static void Main()
    {
        const string outputPath = "backup-2026-05-26.xlsx";
        const string passwordPath = "backup-2026-05-26.password.txt";

        // Generate a strong random password.
        var password = GenerateStrongPassword(20);

        // Build the workbook with some sample data.
        var wb = new Workbook();
        var sheet = wb.AddSheet("Audit log");

        sheet.Cell("A1").Value = "Timestamp";
        sheet.Cell("B1").Value = "Actor";
        sheet.Cell("C1").Value = "Action";
        sheet.Cell("D1").Value = "Resource";
        sheet.Cell("E1").Value = "Outcome";

        for (int i = 0; i < 50; i++)
        {
            sheet.Cell(i + 2, 1).Value = DateTime.Now.AddMinutes(-i);
            sheet.Cell(i + 2, 2).Value = i % 3 == 0 ? "alice@example.com" : "bob@example.com";
            sheet.Cell(i + 2, 3).Value = i % 4 == 0 ? "create" : "update";
            sheet.Cell(i + 2, 4).Value = $"patient/{1000 + i}";
            sheet.Cell(i + 2, 5).Value = i % 7 == 0 ? "denied" : "ok";
        }

        sheet.Columns[1].Width = 20;
        sheet.Columns[2].Width = 24;
        sheet.Columns[3].Width = 12;
        sheet.Columns[4].Width = 16;
        sheet.Columns[5].Width = 10;
        sheet.FreezeRows(1);

        // Save encrypted.
        wb.SaveTo(outputPath, new EncryptionOptions
        {
            Password = password,
            SpinCount = 100_000,
        });

        // Save the password separately. In a real system this goes to a
```

```
// password vault, a separate channel, or a key management service.
File.WriteAllText(passwordPath, password);

Console.WriteLine($"Wrote encrypted {outputPath} ({new
FileInfo(outputPath).Length} bytes)");
Console.WriteLine($"Password stored at {passwordPath} (move to vault)");
}

private static string GenerateStrongPassword(int length)
{
    const string chars =
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz" +
        "0123456789!@#$%^&*()-_+=[]{};:,.<>?";
    var bytes = new byte[length];
    RandomNumberGenerator.Fill(bytes);
    var sb = new StringBuilder(length);
    for (int i = 0; i < length; i++)
        sb.Append(chars[bytes[i] % chars.Length]);
    return sb.ToString();
}
}
```

decrypt-backup.cs

```
C#
using System;
using System.IO;
using Chuvadi.Sheets.Excel;

namespace EncryptedBackupExample;

internal static class Decrypt
{
    private static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.Error.WriteLine("Usage: decrypt-backup <encrypted.xlsx> <password-file>");
            Environment.Exit(2);
        }

        var encryptedPath = args[0];
        var passwordPath = args[1];

        if (!File.Exists(encryptedPath))
        {
            Console.Error.WriteLine($"File not found: {encryptedPath}");
            Environment.Exit(1);
        }

        var password = File.ReadAllText(passwordPath).Trim();

        try
        {
            var wb = Workbook.Load(encryptedPath, password);

            Console.WriteLine($"Decrypted {encryptedPath}.");
            Console.WriteLine($"Sheets: {wb.Sheets.Count}");

            foreach (var sheet in wb.Sheets)
            {
                Console.WriteLine($" {sheet.Name}: {sheet.PopulatedCells.Count()}
cells populated");
            }

            // Optionally re-save unencrypted for processing
            var plainPath = Path.ChangeExtension(encryptedPath, ".decrypted.xlsx");
            wb.SaveTo(plainPath);
            Console.WriteLine($"Wrote decrypted copy to {plainPath}");
        }
        catch (XlsxPasswordRequiredException ex)
        {
            Console.Error.WriteLine($"Password incorrect or missing: {ex.Message}");
            Environment.Exit(1);
        }
        catch (XlsxFormatException ex)
        {
            Console.Error.WriteLine($"Invalid Excel file: {ex.Message}");
            Environment.Exit(1);
        }
    }
}
```

```
        Console.Error.WriteLine($"File is corrupt or not an xlsx: {ex.Message}");  
        Environment.Exit(1);  
    }  
}
```



Appendix D • Complete example — CSV to xlsx pipeline

A pipeline that reads a CSV file, applies some transformations, and writes a styled xlsx output. Demonstrates the streaming writer with an unknown-size source, type inference, and on-the-fly formatting decisions.

C#

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using Chuvadi.Sheets.Excel;

namespace CsvToXlsxExample;

internal static class Program
{
    private static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.Error.WriteLine("Usage: csv-to-xlsx <input.csv> <output.xlsx>");
            Environment.Exit(2);
        }

        var inputPath = args[0];
        var outputPath = args[1];

        using var writer = XlsxWriter.Create(outputPath);
        var sheet = writer.AddSheet("Data");
        sheet.FreezeRows(1);

        var headerStyle = new CellStyleBuilder()
            .Bold().Background("#1A237E").Foreground("#FFFFFF")
            .HAlign(HorizontalAlign.Center).Build();

        var alternateRowStyle = new CellStyleBuilder()
            .Background("#FBF6E8").Build();

        bool isHeader = true;
        int rowIndex = 0;
        int columnCount = 0;

        foreach (var fields in ReadCsv(inputPath))
        {
            if (isHeader)
            {
                columnCount = fields.Count;
                // Set per-column widths now (before first data row)
                for (int i = 0; i < columnCount; i++)
                    sheet.SetColumnWidth(i + 1, EstimateColumnWidth(fields[i]));

                sheet.WriteRow(row =>
                {
                    foreach (var name in fields)
                        row.Cell(name, headerStyle);
                });
                isHeader = false;
                continue;
            }
        }
```

```

        rowIndex++;
        var useAlternate = (rowIndex % 2 == 0);
        sheet.WriteRow(row =>
        {
            foreach (var value in fields)
            {
                var inferred = InferValue(value);
                if (useAlternate && !(inferred is null))
                    row.Cell(inferred, alternateRowStyle);
                else
                    row.Cell(inferred);
            }
        });
    }

    if (rowIndex > 0 && columnCount > 0)
    {
        sheet.AutoFilter($"A1:{ColumnLetter(columnCount)}{rowIndex + 1}");
    }

    writer.Save();
    Console.WriteLine($"Converted {inputPath} → {outputPath} ({rowIndex} data
rows, {columnCount} columns)");
}

// Naive but adequate CSV reader for typical RFC-4180 data
private static IEnumerable<List<string>> ReadCsv(string path)
{
    using var reader = new StreamReader(path);
    string? line;
    while ((line = reader.ReadLine()) is not null)
        yield return SplitCsvLine(line);
}

private static List<string> SplitCsvLine(string line)
{
    var result = new List<string>();
    var current = new System.Text.StringBuilder();
    bool inQuotes = false;
    for (int i = 0; i < line.Length; i++)
    {
        char c = line[i];
        if (inQuotes)
        {
            if (c == '"')
            {
                if (i + 1 < line.Length && line[i + 1] == '"') { cur-
rent.Append('"'); i++; }
                else inQuotes = false;
            }
            else current.Append(c);
        }
        else

```

```

        {
            if (c == ',') { result.Add(current.ToString()); current.Clear(); }
            else if (c == '"' && current.Length == 0) inQuotes = true;
            else current.Append(c);
        }
    }
    result.Add(current.ToString());
    return result;
}

// Type-infer a CSV string into a more specific .NET type
private static object? InferValue(string raw)
{
    if (string.IsNullOrEmpty(raw)) return null;
    var trimmed = raw.Trim();
    if (string.IsNullOrEmpty(trimmed)) return raw;

    if (trimmed.Equals("true", StringComparison.OrdinalIgnoreCase)) return
true;
    if (trimmed.Equals("false", StringComparison.OrdinalIgnoreCase)) return
false;

    if (int.TryParse(trimmed, NumberStyles.Integer, Culture-
Info.InvariantCulture, out var i))
        return i;
    if (long.TryParse(trimmed, NumberStyles.Integer, Culture-
Info.InvariantCulture, out var l))
        return l;
    if (double.TryParse(trimmed, NumberStyles.Float, Culture-
Info.InvariantCulture, out var d))
        return d;

    if (DateTime.TryParse(trimmed, CultureInfo.InvariantCulture, Date-
TimeStyles.None, out var dt))
        return dt;

    return raw;
}

private static double EstimateColumnWidth(string header)
{
    return Math.Min(40, Math.Max(8, header.Length + 4));
}

private static string ColumnLetter(int column1Based)
{
    var result = "";
    while (column1Based > 0)
    {
        column1Based--;
        result = (char)('A' + column1Based % 26) + result;
        column1Based /= 26;
    }
    return result;
}

```

```
}  
}
```

Run with a CSV file of any size — memory stays bounded since the streaming writer flushes rows as they're produced. The CSV reader itself is the only piece that keeps state across rows.



Appendix E · Troubleshooting

Common problems and their resolutions, ordered roughly by how often they come up in practice.

"The file is corrupt or has been changed" in Excel on open

Almost always one of three causes:

1. **Save was not called.** Disposing an `XlsxWriter` without calling `Save` first produces a zero-byte file or a partial file. Always check the file size before assuming success.
2. **Sheet name violations.** Excel rejects sheet names containing `:` `/` `\` `?` `*` `[` `]`, longer than 31 characters, or starting/ending with a single quote. Chuvadi.Sheets validates these at `AddSheet` time, but if you bypass the model (e.g. mutating the streaming writer's internals), the file may pass validation but fail Excel's opening check.
3. **Formula syntax.** A formula with mismatched parentheses or referring to a nonexistent cell will open with an error indicator, but a formula with a structural problem (e.g. trailing operator) may produce the "corrupt" dialog. Test formulas in Excel first if you're unsure.

Dates show as numbers like "45826"

The cell holds a date serial number but has no number format. By default Excel displays this as a large integer. Apply a date format to the cell's style, or rely on the library's default formatting by passing the value as a `DateTime` rather than a `double`.

Shared strings table missing string at index N

This is a Chuvadi.Sheets internal error that should never reach user code. If you see it, file an issue with the input file attached — there is a bug in the placeholder-resolution pass at save time.

"Password is incorrect" but you're sure it isn't

The most common cause is non-ASCII characters in the password. Excel's encryption spec is UTF-16-LE before hashing; Chuvadi.Sheets follows the same. But the source of the password matters — pasting from a Word document with smart quotes, or from a system that normalizes Unicode differently, produces a string that looks identical but hashes to a different value. Strip non-printable characters before passing the password to either save or load.

Reading a file works in Chuvadi but fails in some other library

Chuvadi.Sheets is forgiving — it ignores unknown attributes, fills in missing optional elements, and rebuilds shared-string indices on save. Other libraries may be stricter. If you load a file in Chuvadi, mutate it, save it, and another library cannot open the result, set `StrictXml = true` on the read and re-load the original to see if the source file was already non-conformant.

Encrypted file opens in Excel but Chuvadi cannot load it

This indicates a discrepancy in how Excel-produced encryption is being parsed. As of v1.0, the encryption path has been validated against Excel 2016+ and the spec-strict msoffcrypto-tool reference. If you encounter a file that loads in Excel and fails in Chuvadi, file an issue with the file attached (and a password that can be shared, since the support requires test access).

Memory grows during a long streaming write

Most likely cause: shared-string table growth. Every unique string passes through the SST. If your data has highly varied strings (free-text fields, log messages, IDs), the SST grows roughly linearly with row count. Switch to inline strings:

```
C#
var writer = XlsxWriter.Create("output.xlsx", new XlsxWriterOptions
{
    UseInlineStrings = true,
});
```

This eliminates the SST entirely at the cost of larger output files.

"Sheet shape must be set before the first row is written"

You called `SetColumnWidth`, `FreezeRows`, or `FreezeColumns` after the first `WriteRow`. These methods modify the worksheet's `<sheetViews>` and `<cols>` XML elements, which must be emitted before `<sheetData>`. Restructure your code so all shape settings come first.

"Refusing to extract entry ... (zip-slip)"

The zip archive contains an entry with a path that would resolve outside the target directory. This is almost always malicious or accidental — never expand a zip that triggers this without inspecting the source. To override (only when you fully trust the archive), enumerate entries manually and write them yourself.



Appendix F · Changelog

v1.0.0 — May 2026

Initial public release.

Features:

- Streaming xlsx writer (`XlsxWriter` , `SheetWriter` , `RowBuilder`) with bounded memory
- Streaming xlsx reader (`XlsxReader` , `SheetReader` , `RowReader`) with zero-allocation steady state
- Random-access workbook model (`Workbook` , `Sheet` , `Cell` , `Range`)
- Ergonomic one-liner exports (`.ToXlsx<T>()` , `.ToXlsx()` on `DataTable`)
- Multi-sheet exports via `XlsxExport`
- Ergonomic one-liner reads (`.ReadXlsx()` , `.ReadXlsxSheet()` , `.ReadXlsx<T>()`)
- Cell styling: fonts, fills, borders, number formats, alignment, with fluent builder
- Sheet features: column widths, freeze panes, merge cells, autofilter, hyperlinks, comments
- Structured tables with all built-in Excel styles
- Defined names (workbook-scoped named ranges and scalars)
- Data validation: list, whole number, decimal, date, text length
- Conditional formatting: color scales, data bars, cell-value comparisons
- Sheet protection with per-action permission flags
- Workbook structure protection
- Workbook open-password encryption (OOXML Agile Encryption v4.4, AES-256-CBC, PBKDF2-SHA512, HMAC-SHA512)
- Zip utilities (`ZipWriter` , `ZipReader` , `ZipExtensions`) with path-traversal protection
- Async APIs throughout
- Pure .NET BCL — zero NuGet dependencies

Targets: .NET 10 (compatible with net8 and net9 via `TargetFrameworks` change). MIT licensed.

Validation: Encryption path verified against Microsoft Excel 2016+, LibreOffice Calc 7.x+, and msoffcrypto-tool (spec-strict reference). 33 unit tests pass.



Appendix G · License

MIT License

Copyright (c) 2026 Arun Shivab

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.



Chuvadi.Sheets — The Complete Guide
Version 1.0 — May 2026

सुवडी